

# Data Classification Task Report

목차:

1. 서론
2. EDA
3. Logistic Regression
4. Support Vector Machine
5. Decision Tree
6. Random Forest
7. K-Nearest Neighbors
8. 결론

---

## 1. 서론

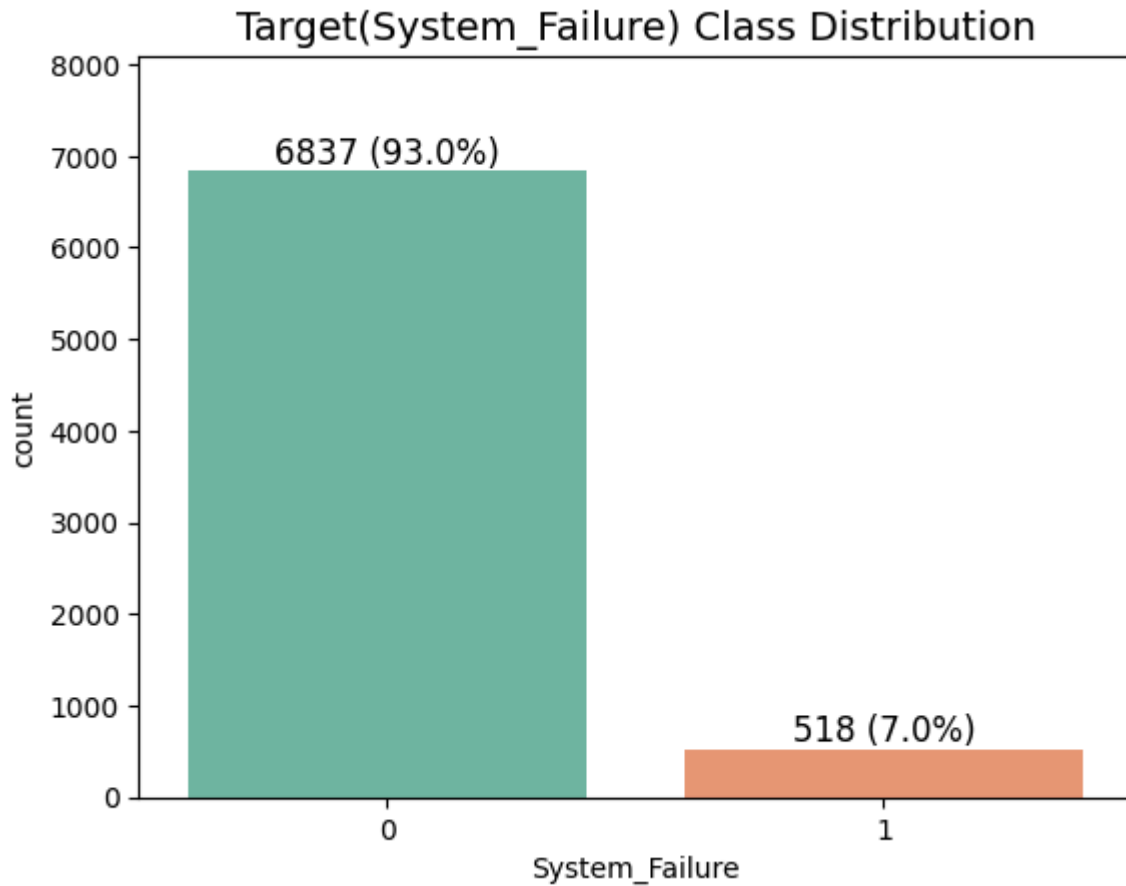
UR3 Cobotops 머신러닝 프로젝트 중 Classification을 실행한다.

주제는 로봇팔의 Robot ProtectiveStop와 grip lost를 종합하여 0과 1의 이산적인 System Failure 피처를 만들어 각 관절들의 Speed, Temperature, Current 피처로 분류해내는것이다.

사용되는 모델은 Logistic Regression (LR), Support Vector Machine (SVM), Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN)이다.

모든 과정은 Standard Scaling을 진행한다.

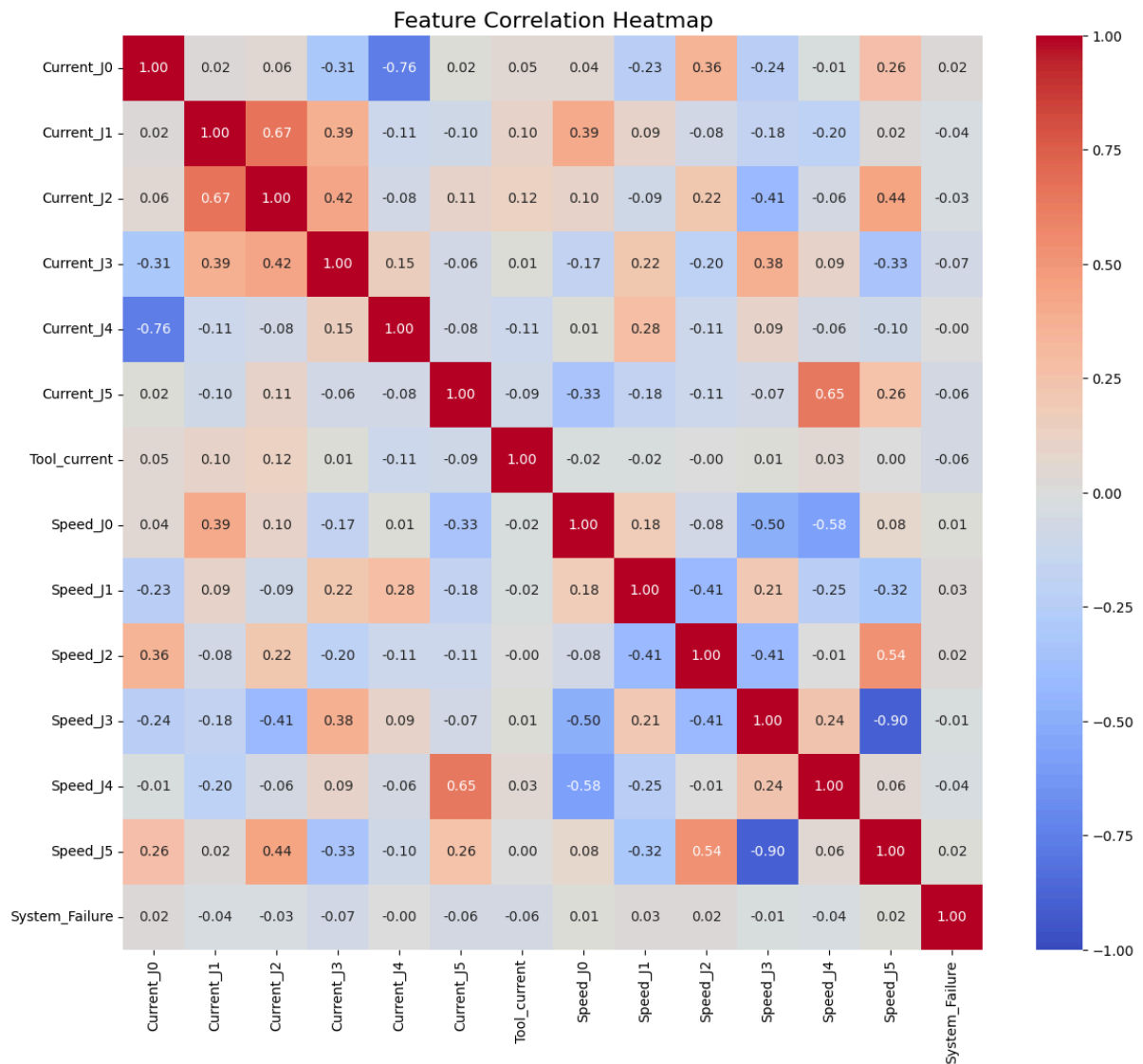
## 2.EDA



Robot ProtectiveStop와 grip lost 중 어떤것이러도 발생했다면 비정상 시스템 정지로 판단하는 **System Failure** 라벨을 만들고 클래스 분포를 보았다.

정상이 93%이고 비정상이 7%인 클래스 불균형을 보인다.

그대로 학습시킨다면 모든 상황을 정상으로만 예측해도 **Accuracy**가 93%를 도달하는 정확도의 역설을 일으키므로, **SMOTE**같은 클래스 불균형을 해소하는 방법을 사용해 고장 데이터를 얼마나 잘 잡았는지 평가하는 **F1-Score**나 **Recall**, **ROC-AUC**를 반드시 확인해야함을 시사한다.



히트맵을 봤을때, 타겟 변수인 **System Failure**과 다른 피쳐들간의 관계가 매우 희미하다.

이는 선형적 관계가 없다는 것을 의미하며 전류, 속도 등이 높아지거나 낮아진다고 해서

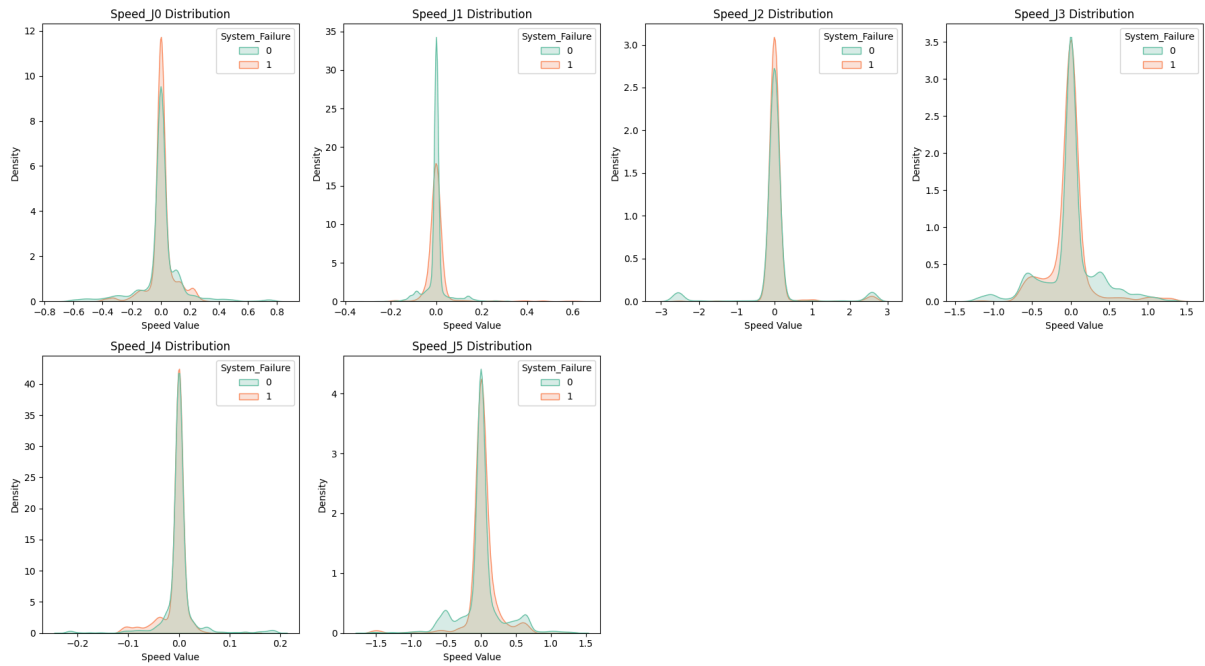
고장 확률이 선형적으로 곧바로 증가하거나 감소하지 않는다.

다만 상관계수는 선형 관계만 측정하므로, 낮은 상관관계가 반드시 예측력이 낮음을

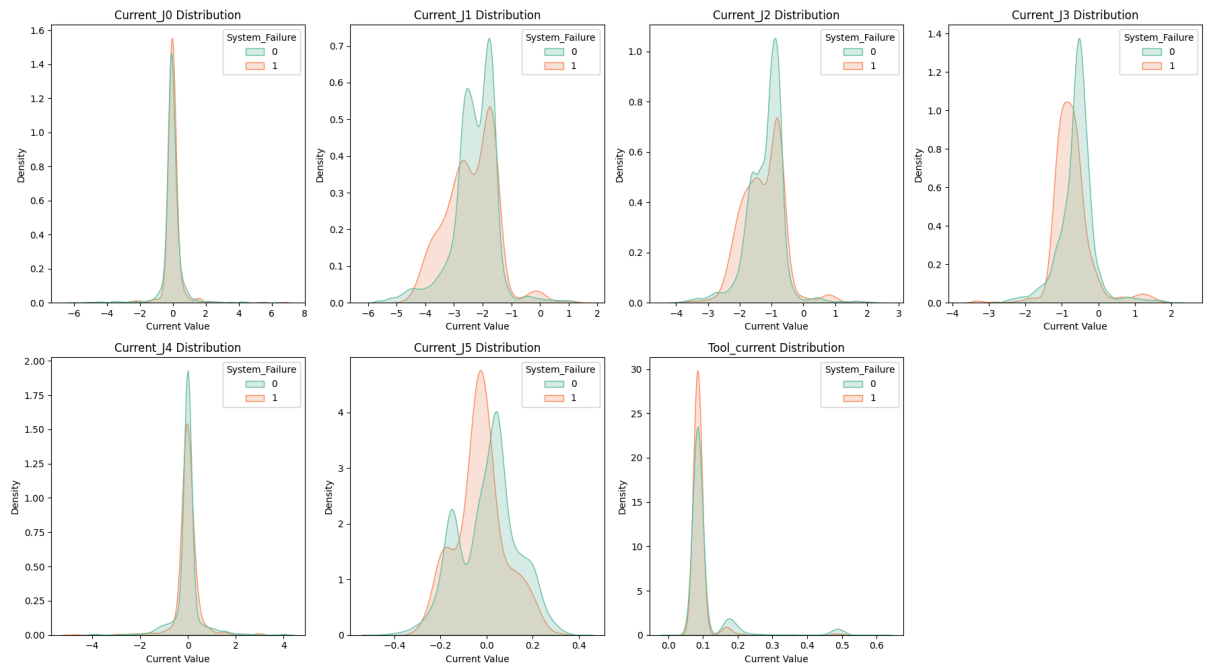
의미하지는 않는다.

따라서 로지스틱 회귀나 **SVM** 같은 모델들보다 트리 기반 앙상블 모델이 더욱 좋은 성능을

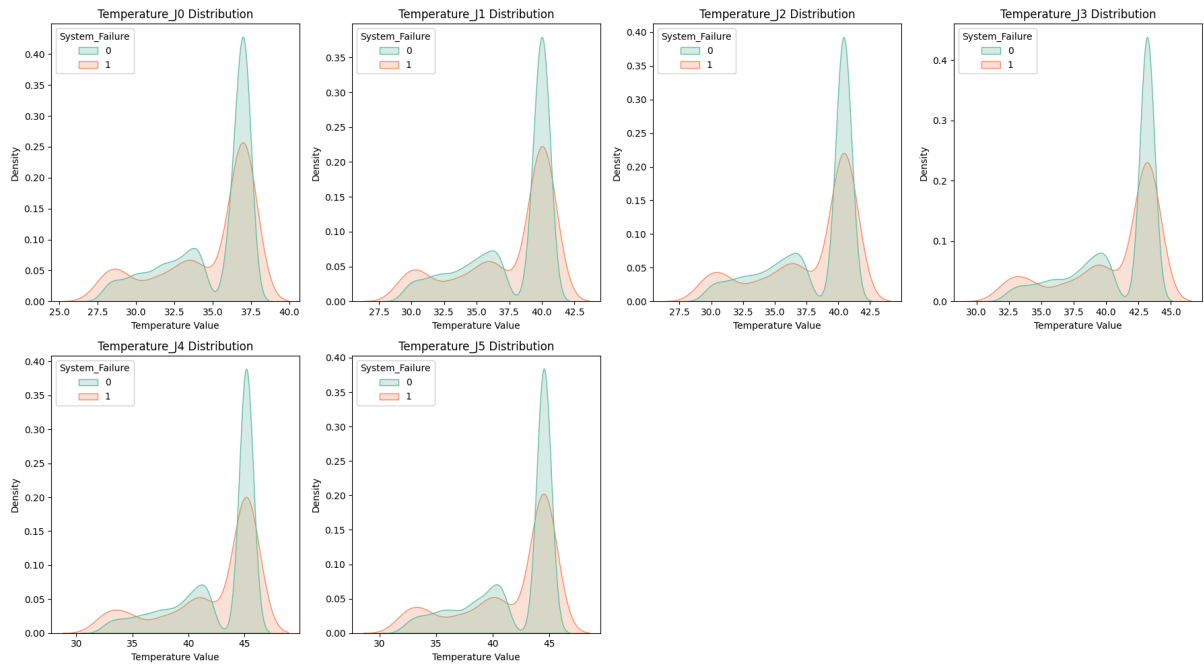
보일것이라 예측된다.



관절별 속도와 고장 여부의 관계를 보면, 정상 분포와 고장 분포가 대부분 겹쳐 있어, 속도 값 자체만으로는 고장 여부를 전혀 판별할 수 없다.



전류를 보면, 평균값은 비슷하지만, 고장 났을 때 데이터가 미세하게 더 넓게 퍼지거나 꼬리가 두꺼워진다.

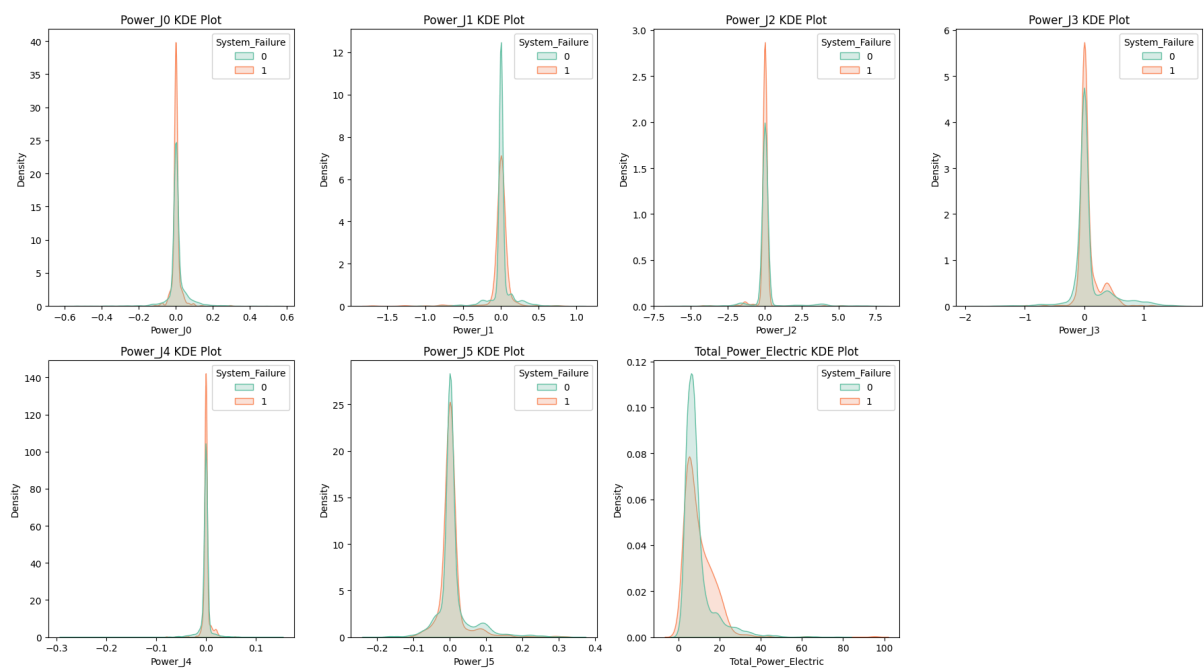


온도를 보면 정상과 고장 상태일 때 데이터가 밀집되는 온도 구간이 확실하게 다르다.

그럼 물리 기반 파생 변수인 **Power**를 만들어서 보자.

$\text{Power} = \text{Current} \times \text{Speed}$  는 속도가 빠르는데 전류도 높게 먹는다면 심한 과부하 상태일 수

있다. 반대로, 속도는 0인데 전류가 높다면 힘을 억지로 주고 있는 고장 전조증상일 수 있다.



Power J0 ~ Power J5는 정상일때와 고장일때의 봉우리의 높이가 다르다. 총 전력량을 보면  
앞선 그래프들과 다르게 오른쪽으로 고장 그래프가 좀 튀어나와있는걸 확인할 수 있었다.

### 3. Logistic Regression

가장 단순하고 해석이 직관적인 로지스틱 회귀로 시작한다.

```
df.columns = df.columns.str.strip()

# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()
df.rename(columns={'Temperature_T0': 'Temperature_J0'}, inplace=True)

# 3. 파생 변수 생성 (Power = Current * Speed 등)
current_cols = [f'Current_J{i}' for i in range(6)]
speed_cols = [f'Speed_J{i}' for i in range(6)]
temp_cols = [f'Temperature_J{i}' for i in range(6)]
for i in range(6):
    df_clean[f'Power_J{i}'] = df_clean[f'Current_J{i}'] * df_clean[f'Speed_J{i}']

df_clean['Total_Power_Electric'] = (df_clean[current_cols]**2).sum(axis=1)

# 4. 입력 피쳐(x) 설정
input_cols = current_cols + speed_cols + temp_cols + ['Tool_current'] + [f'Power_J{i}' for i in range(6)] + ['Total_Power_Electric']
X = df_clean[input_cols]

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개")
✓ 0.0s
```

데이터 크기: (7355, 31)  
피쳐 개수: 26개

온도와 전류, 속도, 파생변수 모두 넣고 학습을 진행한다.

```
# 통합 이진 타겟(System_Failure) 생성: Protective Stop 또는 Grip Lost 발생 시 1
df_clean['Robot_ProtectiveStop'] = df_clean['Robot_ProtectiveStop'].astype(bool).astype(int)
df_clean['grip_lost'] = df_clean['grip_lost'].astype(bool).astype(int)

y = ((df_clean['Robot_ProtectiveStop'] == 1) | (df_clean['grip_lost'] == 1)).astype(int)

print("분류된 타겟(y)의 클래스별 데이터 개수:")
print(y.value_counts().sort_index())
# 0: Normal(정상), 1: Failure(고장)
✓ 0.0s
```

분류된 타겟(y)의 클래스별 데이터 개수:

|   |      |
|---|------|
| 0 | 6837 |
| 1 | 518  |

Name: count, dtype: int64

Protective Stop 또는 Grip Lost 발생 시 1, 아니면 0으로 이진 타겟 라벨을 만든다.

```

# 1. Stratified K-Fold 설정
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Imblearn Pipeline 구축 (Fold 내에서 SMOTE 적용 -> Logistic Regression 학습)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('lr', LogisticRegression(random_state=42, class_weight='balanced', max_iter=1000))
])

# 3. 튜닝할 파라미터 그리드 (Pipeline 사용 시 모델명__파라미터명 형태로 지정)
param_grid = {
    'lr__C': [0.01, 0.1, 1, 10, 100],
    'lr__penalty': ['l1', 'l2'],
    'lr__solver': ['saga']
}

# 4. GridSearchCV 실행 (데이터 불균형을 고려해 scoring을 'f1_macro'로 설정)
print("--- Logistic Regression 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

# 5. 최적 파라미터 및 결과 확인
print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('lr__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

✓ 14.4s

SMOTE를 적용해 오버샘플링을 진행하고 하이퍼파라미터 튜닝을 진행한다. C는 규제 강도의 역수이고 penalty는 L1, L2 방식의 규제 방식을 정한다.

| --- [모델 비교] 상위 10개 Logistic Regression 성능 종합 --- |    |        |          |         |         |           |          |              |                      |
|--------------------------------------------------|----|--------|----------|---------|---------|-----------|----------|--------------|----------------------|
|                                                  | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정              |
| 0                                                | 1  | 0.4819 | 0.4942   | 0.5049  | -0.0108 | 0.642930  | 0.649898 | 0.770230     | C=10, P=I2, S=saga   |
| 1                                                | 2  | 0.4818 | 0.4925   | 0.5045  | -0.0120 | 0.640721  | 0.649218 | 0.770532     | C=1, P=I1, S=saga    |
| 2                                                | 3  | 0.4810 | 0.4948   | 0.5082  | -0.0134 | 0.644290  | 0.653297 | 0.770145     | C=100, P=I1, S=saga  |
| 3                                                | 4  | 0.4809 | 0.4945   | 0.5082  | -0.0137 | 0.643950  | 0.653297 | 0.770068     | C=100, P=I2, S=saga  |
| 4                                                | 5  | 0.4809 | 0.4947   | 0.5082  | -0.0135 | 0.644120  | 0.653297 | 0.770272     | C=10, P=I1, S=saga   |
| 5                                                | 6  | 0.4774 | 0.4887   | 0.5069  | -0.0181 | 0.636472  | 0.656016 | 0.767099     | C=1, P=I2, S=saga    |
| 6                                                | 7  | 0.4655 | 0.4806   | 0.4932  | -0.0126 | 0.623555  | 0.634942 | 0.768886     | C=0.1, P=I1, S=saga  |
| 7                                                | 8  | 0.4639 | 0.4779   | 0.4910  | -0.0131 | 0.619646  | 0.631543 | 0.765278     | C=0.1, P=I2, S=saga  |
| 8                                                | 9  | 0.4549 | 0.4709   | 0.4823  | -0.0114 | 0.606050  | 0.617947 | 0.762872     | C=0.01, P=I2, S=saga |
| 9                                                | 10 | 0.4504 | 0.4570   | 0.4791  | -0.0221 | 0.588375  | 0.608430 | 0.749226     | C=0.01, P=I1, S=saga |

상위 10개의 모델 결과이다. 모든 결과들의 과적합 gap이 거의 없는 수준이며 오히려 Test F1의 값이 더 높다.

모든 모델의 Test F1은 0.5수준이며 Accuracy는 0.65 수준임을 확인 할 수 있다.

모델이 양성과 음성을 서로 얼마나 잘 구별하는지 나타내는 ROC-AUC는 0.77 수준에

위치하여 모델이 정상과 고장을 어느 정도 구분할 줄은 안다는 뜻이다.



```

[선택된 1위 모델의 하이퍼파라미터]
• C: 10
• penalty: l2
• solver: saga

[Train 데이터 평가]
Accuracy (정확도) : 0.6365
F1-Macro Score    : 0.488730
ROC-AUC Score     : 0.715016

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.63   | 0.76     | 5470    |
| 1 (Failure)  | 0.13      | 0.70   | 0.21     | 414     |
| accuracy     |           |        | 0.64     | 5884    |
| macro avg    | 0.55      | 0.67   | 0.49     | 5884    |
| weighted avg | 0.91      | 0.64   | 0.72     | 5884    |

```

[Test 데이터 평가]
Accuracy (정확도) : 0.6560
F1-Macro Score    : 0.506860
ROC-AUC Score     : 0.767099

[Test 상세 분류 리포트]

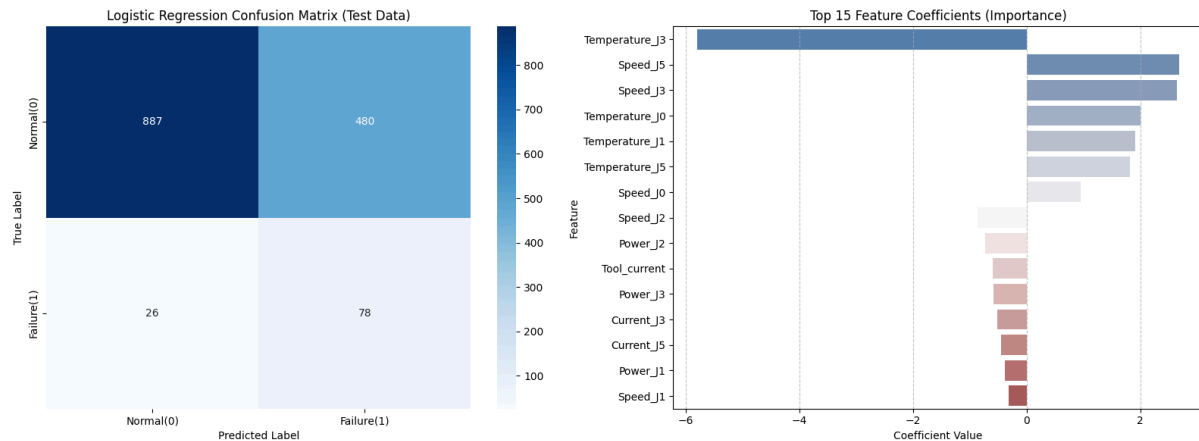
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.65   | 0.78     | 1367    |
| 1 (Failure)  | 0.14      | 0.75   | 0.24     | 104     |
| accuracy     |           |        | 0.66     | 1471    |
| macro avg    | 0.56      | 0.70   | 0.51     | 1471    |
| weighted avg | 0.91      | 0.66   | 0.74     | 1471    |

정상 데이터에서 Precision이 0.97로 모델이 정상이라고 판단한 것 중 97%가 진짜 정상였고, Recall이 0.65으로 실제 정상인 데이터 중 65%만 정상으로 찾아냈다. 고장 데이터에서 Precision은 0.14로 모델이 고장이라고 예측한 것 중 실제 고장은 14%뿐이었다. 너무 많은 정상 데이터를 고장으로 오해하고 있다. Recall은 0.75로 실제 고장 난 데이터 중 75%는 고장이라고 찾아냈다.

**Accuracy**는 65% 정도이지만 데이터가 불균형 상태이므로 신뢰하기 어려운 지표이다.

모델의 균형 잡힌 분류 성능을 보여주는 **F1-Macro Score**도 0.5068으로 현재 매우 낮은 수준이다.



**Confusion Matrix**를 그려본 결과,

실제 정상(0)을 정상(0)으로 예측 (**True Negative**): 887건

실제 정상(0)을 고장(1)으로 예측 (**False Positive**): 480건

실제 고장(1)을 고장(1)으로 예측 (**True Positive**): 78건

실제 고장(1)을 정상(0)으로 예측 (**False Negative**): 26건

으로 나왔다.

원래 고장 데이터가 극도로 적은 불균형 데이터였기에, 이를 해결하기 위해 **SMOTE**로 가짜 고장 데이터를 많이 생성하였다.

모델이 판단 기준을 대폭 낮춰서 조금만 이상해도 고장으로 분류하는 모델이 되어버린 것으로 추측할 수 있다.

또한 정상과 고장 데이터가 선으로 깔끔하게 나뉘지 않고 서로 섞여 있기 때문에 직선 하나만 그을 수 있는 로지스틱 회귀로는 정상 데이터 영역에 수많은 고장 예측 경계가 침범할 수밖에 없다.

변수의 중요도를 분석한 결과로 **Temperature J3** 피처의 온도가 높을수록 오히려 고장 확률이 낮아진다고 보고있다.

**Speed J5, Speed J3, Temperature J0, Temperature J1** 등은 양 방향으로 뻗어있어 해당 속도나 온도가 올라갈수록 모델이 고장이라고 판단한다.

```
=====
--- [L1 규제 분석] 2위 모델의 특성 선택(Feature Selection) 결과 ---
=====

[선택된 2위 모델의 하이퍼파라미터]
  • C: 1
  • penalty: l1
  • solver: saga

🔥 [삭제된 피처] 가중치가 정확히 0이 된 피처: 0개
  - 없음 (모든 피처가 사용되었습니다)

☀️ [살아남은 피처] 가중치가 0이 아닌 피처: 26개 (절대값 크기순 정렬)
  - Temperature_J3      : -10.9191
  - Temperature_J4      : 3.3544
  - Temperature_J1      : 3.2728
  - Speed_J5            : 2.8367
  - Speed_J3            : 2.8211
  - Temperature_J0      : 2.2671
  - Temperature_J5      : 2.0206
  - Speed_J0            : 1.0265
  - Speed_J2            : -0.8672
  - Power_J2            : -0.7284
  - Tool_current        : -0.5914
  ...
  - Current_J4          : 0.0751
  - Current_J1          : 0.0501
  - Total_Power_Electric : -0.0436
  - Temperature_J2      : 0.0000
```

L1으로 규제를 가한 2위와 3위 등의 모델이 어떤 피처를 지웠는지 확인한 결과, 모델들은 모든 피처를 사용하고 0으로 만든 피처는 아무것도 없다.

어째서 이런 결과가 나왔는지는 모델의 순위를 보면 알 수 있다.

상위권 모델의 C는 1 아니면 100으로 규제의 강도가 약했다. 이 말은 특정 변수를 0으로 만들면 성능이 더 떨어졌기에 모든 변수의 미세한 패턴을 다 사용해야 성능이 가장 높게 나오기 때문이라고 할 수 있다.

```
# ENN (Edited Nearest Neighbours) 추가 импорт
from imblearn.under_sampling import EditedNearestNeighbours

# 1. Stratified K-Fold 설정
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# 2. Imblearn Pipeline 구축 (Data Leakage 방지 및 복합 샘플링)
# 흐름: SMOTE(오버샘플링) -> ENN(노이즈 언더샘플링) -> Logistic Regression
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('enn', EditedNearestNeighbours(n_neighbors=3, kind_sel='all')), # ENN 언더샘플링 추가
    ('lr', LogisticRegression(random_state=42, class_weight='balanced', max_iter=1000))
])

# 3. 튜닝할 파라미터 그리드
# Pipeline 사용 시 모델명__파라미터명 형태로 지정합니다.
param_grid = {
    'lr_C': [0.01, 0.1, 1, 10, 100],
    'lr_penalty': ['l1', 'l2'],
    'lr_solver': ['saga']
}

# 4. GridSearchCV 실행
print("--- [SMOTE + 언더샘플링] Logistic Regression 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

# 5. 최적 파라미터 및 결과 확인
print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    {param.replace('lr_', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score :.4f}")
```

SMOTE는 가상의 데이터를 생성하는 과정에서 소수 클래스 주변에 있는 다수 클래스의 영역까지 침범하여 데이터를 생성하는 한계가 존재한다.

이로 인해 클래스 간의 경계선이 모호해지고, 데이터 내에 노이즈가 발생하여 모델이 과적합되거나 분류 경계를 오인하는 부작용이 나타날 수 있다.

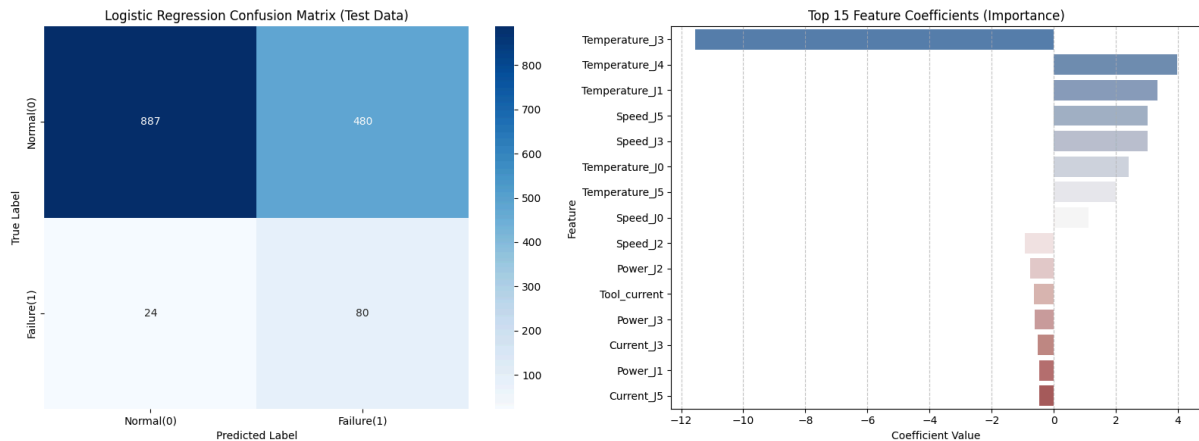
이러한 한계를 극복하고자 **SMOTE**를 통해 생성된 데이터 중, 다른 클래스의 영역에 잘못 침범했거나 경계를 흐리는 노이즈 데이터를 언더샘플링으로 제거해보았다.

즉, 오버샘플링 이후 언더샘플링을 진행했을때의 결과다.

| --- [모델 비교] 상위 10개 Logistic Regression 성능 종합 --- |    |        |          |         |         |           |          |              |                      |
|--------------------------------------------------|----|--------|----------|---------|---------|-----------|----------|--------------|----------------------|
|                                                  | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정              |
| 0                                                | 1  | 0.4872 | 0.5055   | 0.5208  | -0.0153 | 0.664854  | 0.677090 | 0.772417     | C=10, P=I2, S=saga   |
| 1                                                | 2  | 0.4861 | 0.5046   | 0.5222  | -0.0176 | 0.663834  | 0.679130 | 0.772691     | C=100, P=I1, S=saga  |
| 2                                                | 3  | 0.4860 | 0.5051   | 0.5233  | -0.0182 | 0.664174  | 0.679130 | 0.772755     | C=10, P=I1, S=saga   |
| 3                                                | 4  | 0.4860 | 0.5048   | 0.5229  | -0.0181 | 0.663664  | 0.678450 | 0.772684     | C=100, P=I2, S=saga  |
| 4                                                | 5  | 0.4853 | 0.5062   | 0.5188  | -0.0126 | 0.665874  | 0.675731 | 0.772677     | C=1, P=I1, S=saga    |
| 5                                                | 6  | 0.4797 | 0.5027   | 0.5208  | -0.0181 | 0.660945  | 0.677090 | 0.769294     | C=1, P=I2, S=saga    |
| 6                                                | 7  | 0.4668 | 0.4975   | 0.5125  | -0.0150 | 0.652447  | 0.666213 | 0.769702     | C=0.1, P=I1, S=saga  |
| 7                                                | 8  | 0.4655 | 0.4940   | 0.5099  | -0.0159 | 0.646839  | 0.665534 | 0.766438     | C=0.1, P=I2, S=saga  |
| 8                                                | 9  | 0.4578 | 0.4919   | 0.5085  | -0.0166 | 0.646159  | 0.660095 | 0.762225     | C=0.01, P=I2, S=saga |
| 9                                                | 10 | 0.4553 | 0.4889   | 0.5067  | -0.0178 | 0.645309  | 0.657376 | 0.745315     | C=0.01, P=I1, S=saga |

교차검증(CV), Train, Test 모든 구간에서 **F1-Score**와 **Accuracy**가 동반 상승하였다.

다수 클래스 데이터 중 소수 클래스 무리에 너무 깊숙이 침투해 있어 모델을 헛갈리게 만드는 경계선 또는 노이즈 데이터를 삭제하면서 결과적으로 경계선이 깔끔해지면서 모델이 정상과 고장을 구분하는 선을 긋기가 훨씬 수월해졌고 오탐지가 줄어들어 **F1-Score**와 **Accuracy**가 동시에 상승한 것으로 보인다.



Confusion Matrix를 살펴보면 실제 고장인 TP를 2건 더 찾아냈고 놓친 고장인 FP는 2건 줄었다.

하지만 FP는 단 1건도 줄지 않았다. 즉, 앞선 EDA에서 봤듯이 데이터셋은 직선으로는 절대 풀 수 없는 복잡한 비선형 구조를 띄고있음을 증명한다.

언더샘플링을 적용해 경계선을 깔끔히 만들었을때 변수 중요도가 크게 변했다.

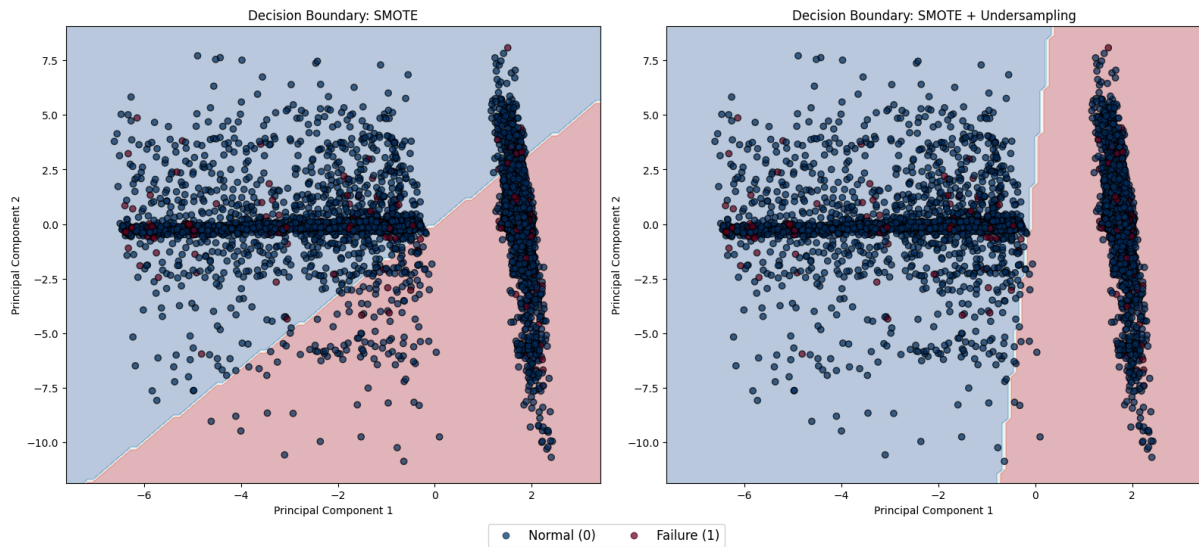
그래프의 X축 스케일을 보면 SMOTE 그래프는 대략 -6 ~ +3 사이였던 반면 언더샘플링을 하고 난 후의 그래프는 -12 ~ +4 이상으로 급격하게 커졌다.

변수들의 순위는 비슷하지만, 모델이 각 변수에 부여하는 확신이 비정상적으로 커졌다.

이유로는 고장과 정상 데이터가 서로 맞닿아 헛갈리는 구역의 데이터를 지워버렸기에 두 클래스 사이에 빈 공간이 생긴다.

따라서 모델 입장에서는 경계선 근처에 방해물이 사라졌으니, 잘 분류해냈다고 착각하게 된다.

이 확신이 수학적으로는 계수의 크기 증가로 나타났다고 추측한다.



26개의 피처를 2차원으로 PCA 한 후 그려본 Decision Boundary를 보면 파란색 점이 정상, 빨간색 점이 고장이다.

빨간 점들은 거대한 파란 점 무리 속에 완벽하게 파묻혀 있다. PCA 2차원 공간에서도 정상과 고장 데이터가 상당 부분 중첩되어 나타났으며, 이는 선형 분리가 쉽지 않음을 보여준다.

왼쪽 그래프는 SMOTE만 적용했을 때 그은 결정 경계 그래프이다. 억지로 늘어난 빨간 점들을 어떻게든 많이 맞추려고 오른쪽 파란 기둥과 왼쪽 파란 뭉치의 하단을 대각선으로 분단해 빨간색 배경 안에 갇혀있는 수많은 파란 점들이 오차 행렬에서 보았던 480개의 False Positive라고 할 수 있다.

오른쪽 그래프는 SMOTE에서 언더샘플링을 적용해 경계선 부분의 데이터를 지우고 그은 결정 경계 그래프이다. 헛갈리는 경계면의 데이터를 지웠더니, 모델이 결정 경계를 대각선에서 수직으로 그었다.

대각선으로 그으나, 수직으로 그으나, 결국 뭉쳐있는 파란 점들을 엄청나게 썰어낼 수밖에 없다. 왼쪽에서 대각선에 잘려 나갔던 파란 점의 개수나, 오른쪽에서 수직선에 갇혀버린

파란 점의 개수가 비슷하게 480여 개였기 때문에, 오차 행렬의 FP 숫자가 단 1건도

줄어들지 않고 똑같았던 것으로 추측할 수 있다.

언더샘플링 이후 특정 변수의 가중치가 갑자기 폭등했던 이유는 모델이 이렇게 판단

기준선의 방향을 완전히 틀어버렸기 때문에, 중요하게 생각하는 변수의 순위도 요동쳤던

것이다.

## 4. Support Vector Machine

LR의 실패 원인은 직선으로 나눌 수 없는 데이터 구조 때문이라는 것을 잘 확인했다.

SVM은 RBF 커널이라는 방법을 써서 2차원/3차원 데이터를 고차원 데이터로 구부려서

분류한다.

만약 RBF SVM의 성능이 LR보다 확연히 좋고 FP가 줄어든다면, 비선형 모델인 DT나 RF의

성능도 기대해볼 수 있다.

```
df.columns = df.columns.str.strip()
df.rename(columns={'Temperature_T0': 'Temperature_J0'}, inplace=True)

# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()

# 3. 파생 변수 생성 (Power = Current * Speed 등)
current_cols = [f'Current_J{i}' for i in range(6)]
speed_cols = [f'Speed_J{i}' for i in range(6)]
temp_cols = [f'Temperature_J{i}' for i in range(6)]
for i in range(6):
    df_clean[f'Power_J{i}'] = df_clean[f'Current_J{i}'] * df_clean[f'Speed_J{i}']

df_clean['Total_Power_Electric'] = (df_clean[current_cols]**2).sum(axis=1)

# 4. 입력 피쳐(X) 설정
input_cols = current_cols + speed_cols + temp_cols + ['Tool_current'] + [f'Power_J{i}' for i in range(6)] + ['Total_Power_Electric']
X = df_clean[input_cols]

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개")
✓ 0.0s

데이터 크기: (7355, 31)
피쳐 개수: 26개
```

26개의 피쳐를 모두 사용한다.



```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축
# probability=True: ROC-AUC 계산 및 확률 출력을 위해 필요 (단, 학습 시간이 조금 늘어남)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('svc', SVC(random_state=42, probability=True))
])

# 파라미터 그리드 (비선형 rbf 커널과 선형 linear 커널 비교)
param_grid = {
    'svc__C': [0.1, 1, 10],          # 규제 강도 (오차 허용 범위)
    'svc__gamma': ['scale', 'auto', 0.1], # 결정 경계의 곡률 (rbf 커널용)
    'svc__kernel': ['rbf', 'linear']    # 커널 종류
}

print("--- SVM 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 (시간이 다소 소요됩니다) ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('svc__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

SMOTE를 사용해 클래스 불균형을 해소하고 gridsearch를 진행한다.

C는 규제의 역수, kernel의 종류는 선형 커널인 linear, 저차원에서는 선형 분리가 불가능한

데이터를 고차원 공간으로 매핑하는 RBF를 사용한다.

--- [모델 비교] 상위 10개 SVM 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                 |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|-------------------------|
| 0 | 1  | 0.6808 | 0.7664   | 0.7258  | 0.0406  | 0.911285  | 0.900748 | 0.860774     | C=10, G=scale, K=rbf    |
| 1 | 2  | 0.6804 | 0.7825   | 0.7112  | 0.0713  | 0.917573  | 0.889871 | 0.868859     | C=10, G=0.1, K=rbf      |
| 2 | 3  | 0.6776 | 0.7495   | 0.7285  | 0.0210  | 0.901598  | 0.899388 | 0.860883     | C=1, G=0.1, K=rbf       |
| 3 | 4  | 0.6759 | 0.7526   | 0.7258  | 0.0268  | 0.904657  | 0.900748 | 0.863809     | C=10, G=auto, K=rbf     |
| 4 | 5  | 0.6619 | 0.7088   | 0.7063  | 0.0025  | 0.883923  | 0.889871 | 0.863900     | C=1, G=scale, K=rbf     |
| 5 | 6  | 0.6599 | 0.7004   | 0.7079  | -0.0076 | 0.878824  | 0.889871 | 0.864498     | C=1, G=auto, K=rbf      |
| 6 | 7  | 0.6532 | 0.6934   | 0.6929  | 0.0006  | 0.877804  | 0.887831 | 0.856754     | C=0.1, G=0.1, K=rbf     |
| 7 | 8  | 0.6186 | 0.6443   | 0.6676  | -0.0233 | 0.840075  | 0.859959 | 0.847497     | C=0.1, G=scale, K=rbf   |
| 8 | 9  | 0.6163 | 0.6390   | 0.6676  | -0.0287 | 0.835656  | 0.858600 | 0.847645     | C=0.1, G=auto, K=rbf    |
| 9 | 10 | 0.4934 | 0.5159   | 0.5341  | -0.0181 | 0.679470  | 0.690007 | 0.784783     | C=10, G=scale, K=linear |

상위 10개의 모델들을 보면, 1위부터 9위까지 RBF 커널을 사용했다. 유일하게 Linear를

사용한 10위 모델은 LR의 결과와 거의 유사하게 나왔다.

나머지 모델들은 F1, Accuracy, ROC-AUC 모두 크게 상승했다.

이전 LR의 최고의 모델과 현재 1위 SVM 모델을 비교하면 F1 score는 0.2 이상 상승했고 약 67%였던 Accuracy가 90% 이상까지 상승했다. 또한 ROC-AUC는 약 0.09가 상승했다.

```

[선택된 3위 모델의 하이퍼파라미터]
  • C: 1
  • gamma: 0.1
  • kernel: rbf

[Train 데이터 평가]
Accuracy (정확도) : 0.9016
F1-Macro Score    : 0.749483
ROC-AUC Score     : 0.932674

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.99      | 0.90   | 0.94     | 5470    |
| 1 (Failure)  | 0.41      | 0.87   | 0.55     | 414     |
| accuracy     |           |        | 0.90     | 5884    |
| macro avg    | 0.70      | 0.89   | 0.75     | 5884    |
| weighted avg | 0.95      | 0.90   | 0.92     | 5884    |

```

[Test 데이터 평가]
Accuracy (정확도) : 0.8994
F1-Macro Score    : 0.728527
ROC-AUC Score     : 0.860883

[Test 상세 분류 리포트]

```

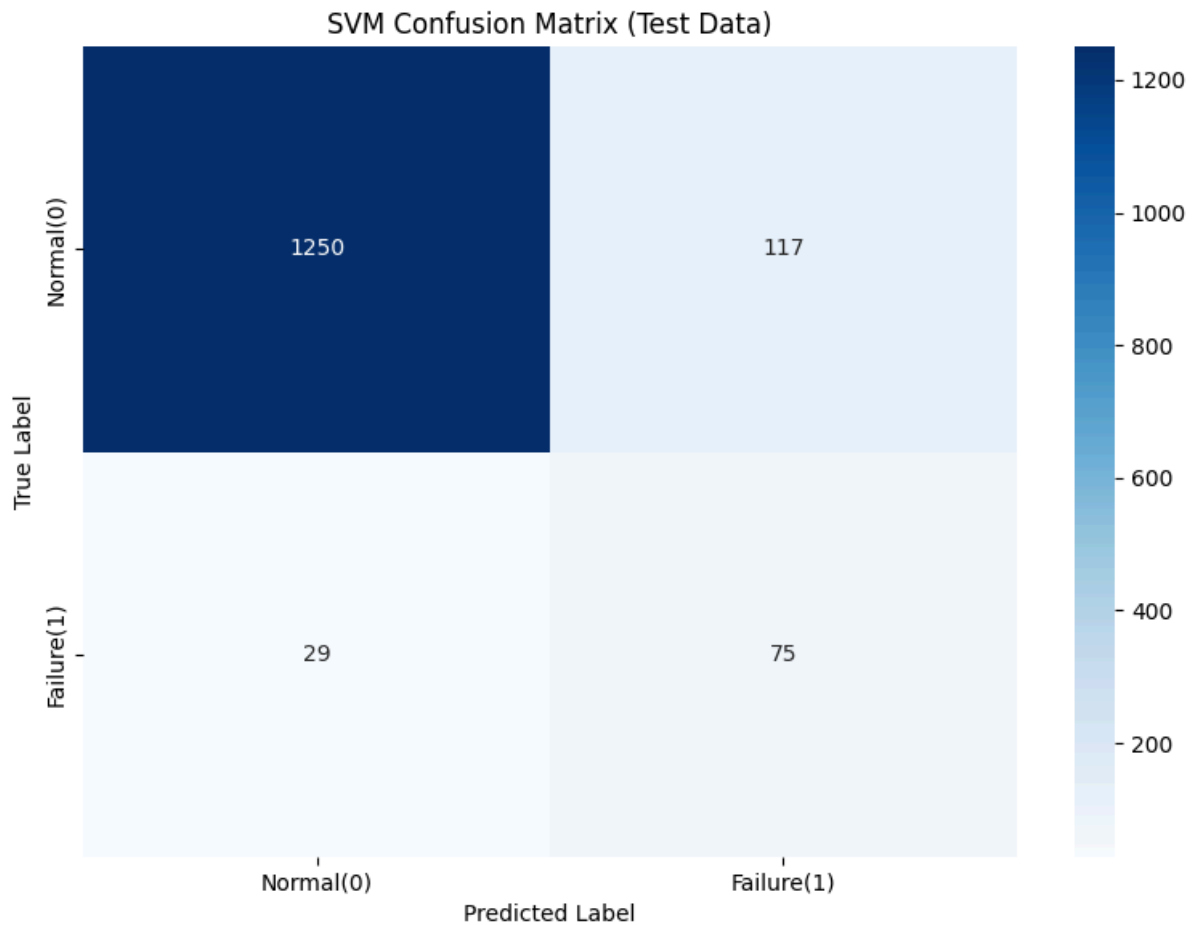
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.98      | 0.91   | 0.94     | 1367    |
| 1 (Failure)  | 0.39      | 0.75   | 0.51     | 104     |
| accuracy     |           |        | 0.90     | 1471    |
| macro avg    | 0.68      | 0.83   | 0.73     | 1471    |
| weighted avg | 0.94      | 0.90   | 0.91     | 1471    |

가장 Test F1이 높고 과적합 Gap이 낮은 3위 모델의 Failure 세부 지표를 보면

Precision은 0.39로 전보다 약 2.8배 높아졌다.

Recall은 전과 같은 0.75를 기록했다.

F1 score는 0.51로 2배 이상 높아졌다.



오차 행렬을 보면 **False Positive**가 117건으로 480건이던 LR에 비해 약 **75%**나 줄어들었다.

또한 **True Negative**가 그만큼 증가하여 1250건이 되었다. **Accuracy**가 약 **90%**까지 증가한 이유이다.

**True Positive**는 75건으로 80건이던 LR에 비해 5건을 놓쳤다. 하지만 **FP**를 363건이나 줄이는 성과를 얻기위해 **TP**가 소폭 감소했으나 **FP**가 크게 감소하여 **Precision**이 크게 향상되었다.

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정               |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|-----------------------|
| 0 | 1  | 0.6873 | 0.7845   | 0.7185  | 0.0659  | 0.920122  | 0.895989 | 0.866151     | C=10, G=0.1, K=rbf    |
| 1 | 2  | 0.6806 | 0.7545   | 0.7317  | 0.0228  | 0.905846  | 0.901428 | 0.855192     | C=1, G=0.1, K=rbf     |
| 2 | 3  | 0.6799 | 0.7612   | 0.7328  | 0.0284  | 0.910605  | 0.904147 | 0.862100     | C=10, G=auto, K=rbf   |
| 3 | 4  | 0.6796 | 0.7705   | 0.7322  | 0.0382  | 0.914684  | 0.904827 | 0.861263     | C=10, G=scale, K=rbf  |
| 4 | 5  | 0.6682 | 0.7133   | 0.7122  | 0.0011  | 0.888001  | 0.893950 | 0.863760     | C=1, G=scale, K=rbf   |
| 5 | 6  | 0.6667 | 0.7021   | 0.7122  | -0.0101 | 0.881883  | 0.893950 | 0.864027     | C=1, G=auto, K=rbf    |
| 6 | 7  | 0.6553 | 0.6969   | 0.6939  | 0.0030  | 0.882563  | 0.889871 | 0.855221     | C=0.1, G=0.1, K=rbf   |
| 7 | 8  | 0.6241 | 0.6514   | 0.6700  | -0.0186 | 0.847383  | 0.864718 | 0.847040     | C=0.1, G=scale, K=rbf |
| 8 | 9  | 0.6207 | 0.6472   | 0.6708  | -0.0236 | 0.844663  | 0.864038 | 0.846710     | C=0.1, G=auto, K=rbf  |
| 9 | 10 | 0.5101 | 0.5279   | 0.5539  | -0.0260 | 0.699184  | 0.721278 | 0.781554     | C=1, G=0.1, K=linear  |

SMOTE의 단점을 보완하기 위해 언더샘플링을 추가적으로 진행했다.

가장 성능이 좋은 모델은 3위 모델로 F1 스코어가 0.73이다.

```

[선택된 3위 모델의 하이퍼파라미터]
• C: 10
• gamma: auto
• kernel: rbf

[Train 데이터 평가]
Accuracy (정확도) : 0.9089
F1-Macro Score    : 0.758758
ROC-AUC Score     : 0.936775

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.99      | 0.91   | 0.95     | 5470    |
| 1 (Failure)  | 0.43      | 0.85   | 0.57     | 414     |
| accuracy     |           |        | 0.91     | 5884    |
| macro avg    | 0.71      | 0.88   | 0.76     | 5884    |
| weighted avg | 0.95      | 0.91   | 0.92     | 5884    |

```

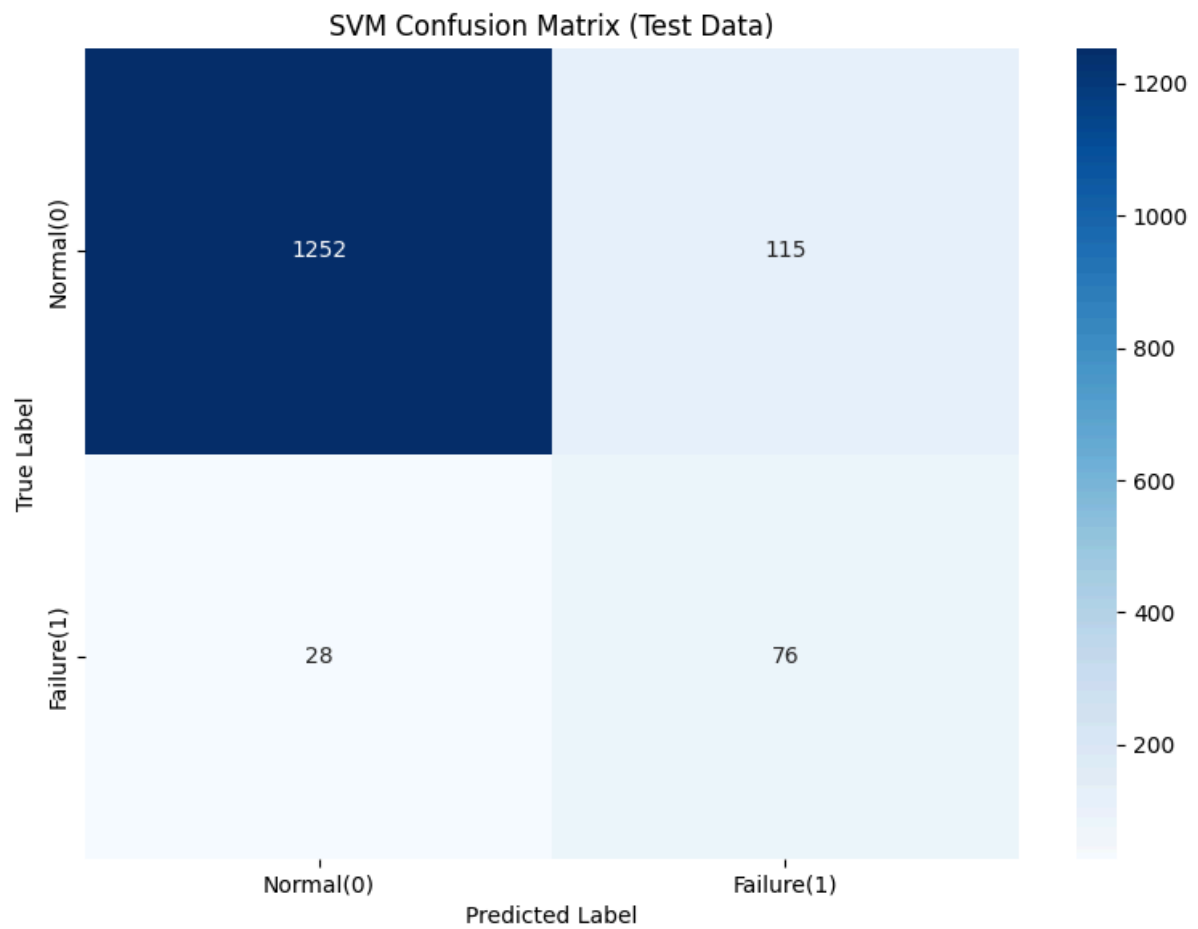
[Test 데이터 평가]
Accuracy (정확도) : 0.9028
F1-Macro Score    : 0.730615
ROC-AUC Score     : 0.861952

[Test 상세 분류 리포트]

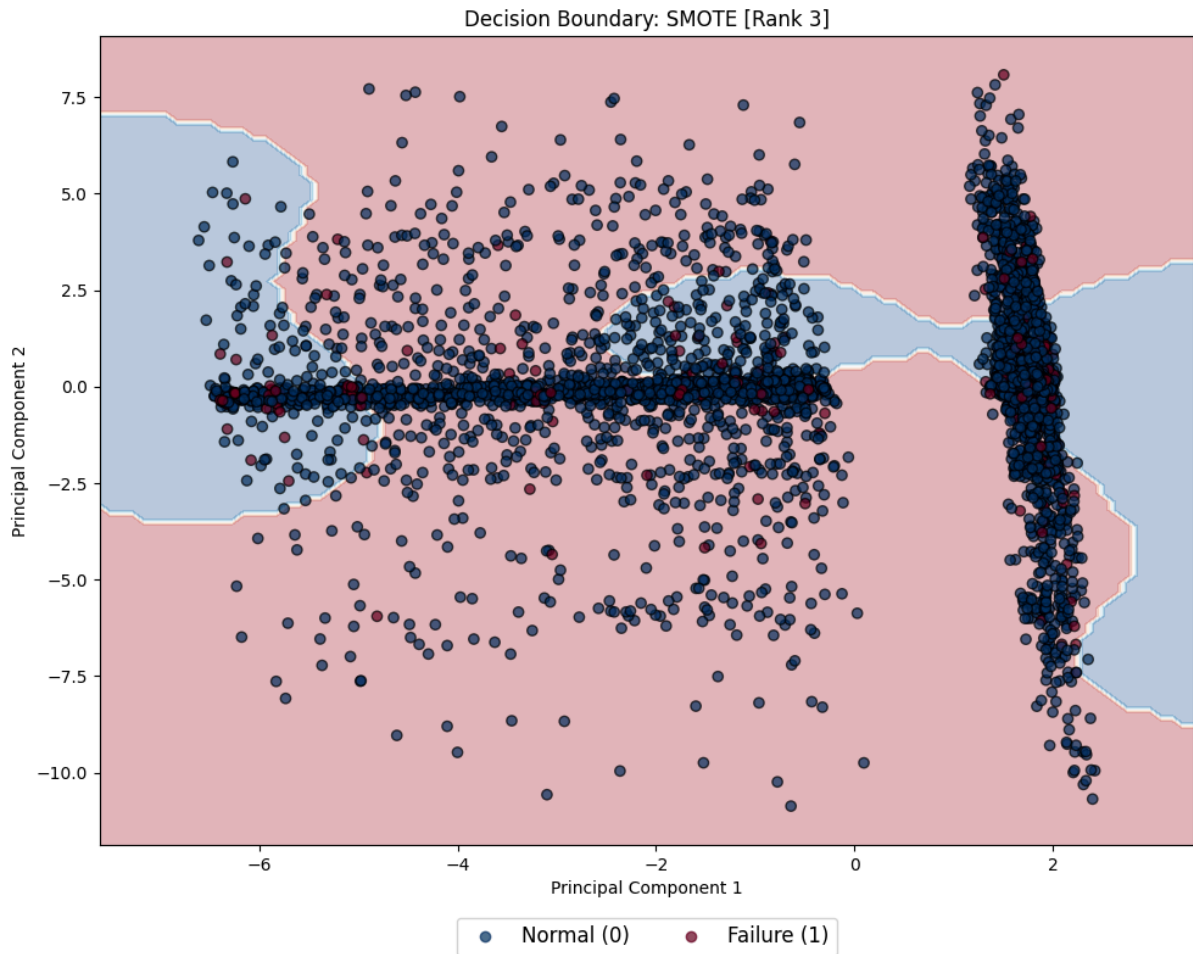
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.98      | 0.92   | 0.95     | 1367    |
| 1 (Failure)  | 0.40      | 0.73   | 0.52     | 104     |
| accuracy     |           |        | 0.90     | 1471    |
| macro avg    | 0.69      | 0.82   | 0.73     | 1471    |
| weighted avg | 0.94      | 0.90   | 0.92     | 1471    |

세부 지표를 보면 그냥 SMOTE를 한것에 비해 Precision과 f1 score가 0.01 올랐지만 Recall은 0.02 감소했다. 전형적인 Trade off로 SMOTE ENN이 모든면에서 우수한 결과를 내지는 못했다.



오차행렬을 보면 TN이 1252건, FP가 115건, FN가 28건, TP가 76건이다.

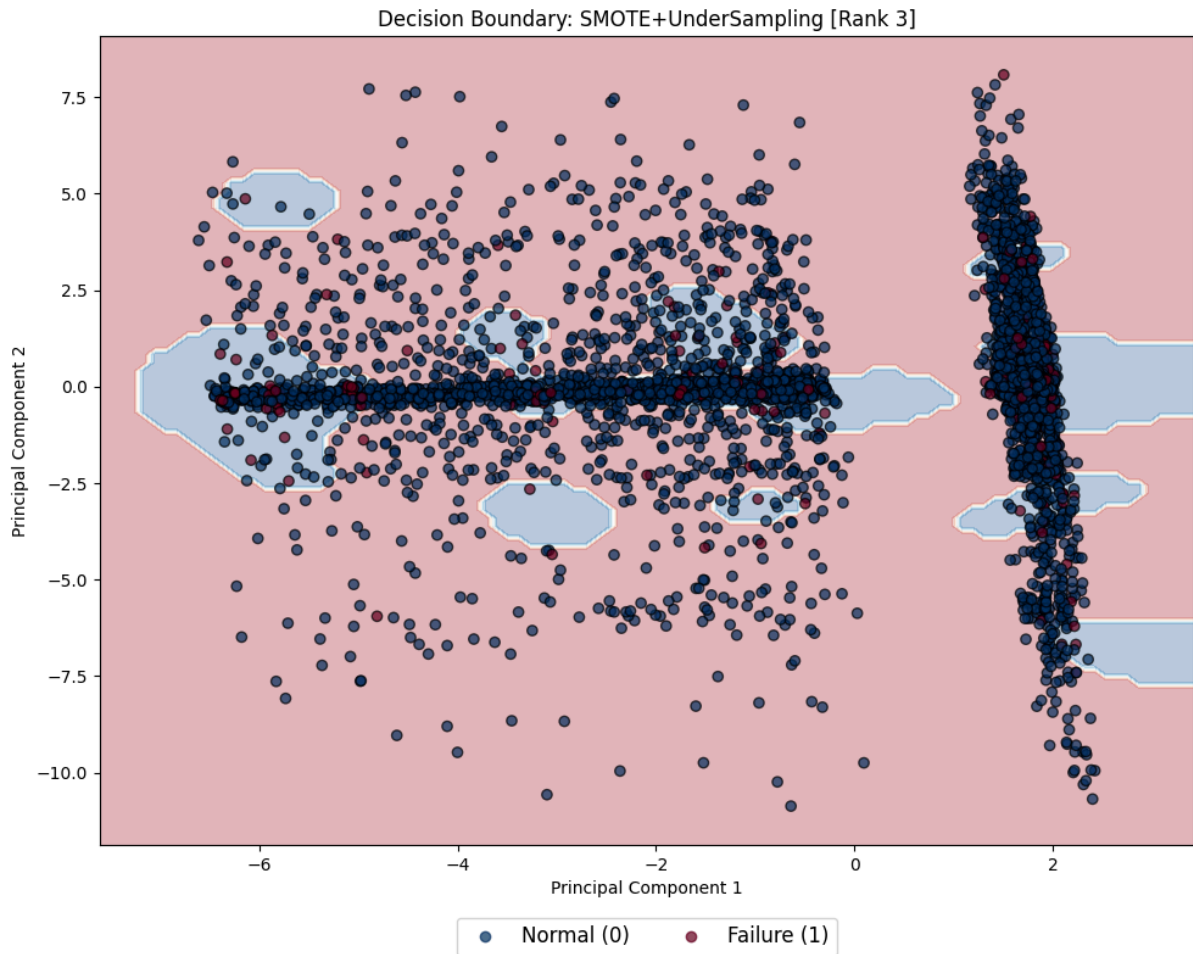


SMOTE의 가장 좋은 모델의 Decision Boundary를 보면 넓고 큼직한 섬 형태의 경계선이 그려져있다.

왜냐하면 **SMOTE**는 가짜 고장 데이터를 무작위로 생성하기 때문에 경계면이 지저분하고 노이즈가 많다.

따라서 모델은 **C=1**이라는 규제를 주어 더 일반적이고 부드러운 경계선을 그었다.





SMOTE에 언더샘플링을 적용한 가장 Test F1이 높았던 모델의 Decision Boundary를 보면, 언더샘플링을 통해 경계선의 지지분한 데이터를 삭제했기 때문에  $C = 10$ 이라는 약한 규제를 적용해 복잡하고 작은 여러 섬 모양으로 그려져있다.

## 5. Decision Tree

앞서 확인한 LR과 SVM의 결과를 통해, 본 데이터셋은 선형 경계로는 정상과 고장을 명확히 분류할 수 없는 복잡한 비선형 구조를 띠고 있음을 입증했다. RBF 커널을 통해 비선형성을 부여한 SVM이 FP를 대폭 줄이며 성능을 향상시킨 것처럼, 비선형적 패턴을 학습하는 트리 기반 모델 역시 우수한 성능을 보일 것으로 예상된다.

또한 DT는 모델의 의사결정 과정을 시각적으로 확인할 수 있다. 앞서 생성한 파생 변수나 특정 온도, 속도 피쳐들이 실제 분기점에서 얼마나 중요하게 작용하는지 확인해 볼 수 있을 것이다.

```
df.columns = df.columns.str.strip()
df.rename(columns={'Temperature_T0': 'Temperature_J0'}, inplace=True)

# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()

# 3. 파생 변수 생성 (Power = Current * Speed 등)
current_cols = [f'Current_J{i}' for i in range(6)]
speed_cols = [f'Speed_J{i}' for i in range(6)]
temp_cols = [f'Temperature_J{i}' for i in range(6)]
for i in range(6):
    df_clean[f'Power_J{i}'] = df_clean[f'Current_J{i}'] * df_clean[f'Speed_J{i}']

df_clean['Total_Power_Electric'] = (df_clean[current_cols]**2).sum(axis=1)

# 4. 입력 피쳐(X) 설정
input_cols = current_cols + speed_cols + temp_cols + ['Tool_current'] + [f'Power_J{i}' for i in range(6)] + ['Total_Power_Electric']
X = df_clean[input_cols]

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개")
```

✓ 0.0s P

데이터 크기: (7355, 31)  
피쳐 개수: 26개

LR, SVM처럼 26개의 피쳐를 모두 사용한다.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('dt', DecisionTreeClassifier(random_state=42))
])

# 파라미터 그리드 (트리 깊이 및 분할 조건 탐색)
param_grid = {
    'dt__criterion': ['gini', 'entropy'],
    'dt__max_depth': [5, 7, 10, 15],
    'dt__min_samples_leaf': [7, 9, 11],
    'dt__min_samples_split': [10, 15, 20]
}

print("--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('dt__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

클래스 불균형을 해소하기 위해 SMOTE를 사용하고 Grid Search를 진행한다.

각 파라미터의 의미는 다음과 같다.

**criterion**는 분할 품질을 측정하는 기능으로 **gini**라면 지니 불순도를 기반, **entropy**라면 IG를 기반으로 분할한다.

**max depth**은 트리의 깊이를 제한하는 파라미터이다.

**min samples leaf**는 리프 노드가 되기 위해 필요한 최소 샘플 수를 정하는 파라미터이다.

min samples split은 노드를 분할하기 위해 필요한 최소 샘플 수를 정하는 파라미터이다.

| --- [모델 비교] 상위 10개 Decision Tree 성능 종합 --- |    |        |          |         |         |           |          |              |                        |
|--------------------------------------------|----|--------|----------|---------|---------|-----------|----------|--------------|------------------------|
|                                            | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                |
| 0                                          | 1  | 0.6805 | 0.8458   | 0.7022  | 0.1436  | 0.950034  | 0.899388 | 0.811610     | C=ent, D=15, L=7, S=15 |
| 1                                          | 2  | 0.6794 | 0.8494   | 0.7071  | 0.1424  | 0.951394  | 0.900068 | 0.821581     | C=ent, D=15, L=7, S=10 |
| 2                                          | 3  | 0.6740 | 0.8276   | 0.6921  | 0.1355  | 0.942386  | 0.892590 | 0.823930     | C=ent, D=15, L=7, S=20 |
| 3                                          | 4  | 0.6684 | 0.8239   | 0.6835  | 0.1403  | 0.940347  | 0.885112 | 0.817997     | C=ent, D=15, L=9, S=20 |
| 4                                          | 5  | 0.6678 | 0.8281   | 0.6911  | 0.1370  | 0.942726  | 0.890551 | 0.819168     | C=ent, D=15, L=9, S=10 |
| 5                                          | 5  | 0.6678 | 0.8281   | 0.6911  | 0.1370  | 0.942726  | 0.890551 | 0.819168     | C=ent, D=15, L=9, S=15 |
| 6                                          | 7  | 0.6609 | 0.7760   | 0.6749  | 0.1010  | 0.914684  | 0.867437 | 0.826691     | C=gin, D=15, L=7, S=15 |
| 7                                          | 8  | 0.6596 | 0.7778   | 0.6782  | 0.0996  | 0.915024  | 0.867437 | 0.824359     | C=gin, D=15, L=7, S=10 |
| 8                                          | 9  | 0.6582 | 0.7632   | 0.6800  | 0.0833  | 0.908566  | 0.870156 | 0.827651     | C=gin, D=15, L=9, S=15 |
| 9                                          | 9  | 0.6582 | 0.7632   | 0.6800  | 0.0833  | 0.908566  | 0.870156 | 0.827651     | C=gin, D=15, L=9, S=10 |

CV 값과 Test Accuracy는 SVM과 비슷하지만 F1과 ROC-AUC 값은 SVM에 비해 오히려 하락하였다.

또한 SVM에 비해 과적합 Gap이 많이 늘어났다. 가장 좋은 모델인 2위 모델의 과적합 Gap은 0.1424이다.

하이퍼파라미터 튜닝 결과

| --- [모델 비교] 상위 10개 Decision Tree 성능 종합 --- |    |        |          |         |         |           |          |              |                         |
|--------------------------------------------|----|--------|----------|---------|---------|-----------|----------|--------------|-------------------------|
|                                            | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                 |
| 0                                          | 1  | 0.6348 | 0.7172   | 0.6450  | 0.0721  | 0.881713  | 0.850442 | 0.817561     | C=ent, D=10, L=7, S=15  |
| 1                                          | 2  | 0.6340 | 0.7234   | 0.6400  | 0.0834  | 0.887492  | 0.849082 | 0.815110     | C=ent, D=10, L=7, S=10  |
| 2                                          | 3  | 0.6311 | 0.7155   | 0.6440  | 0.0715  | 0.881033  | 0.851122 | 0.821714     | C=ent, D=10, L=7, S=20  |
| 3                                          | 4  | 0.6284 | 0.7161   | 0.6436  | 0.0726  | 0.881713  | 0.849082 | 0.821440     | C=ent, D=10, L=9, S=20  |
| 4                                          | 5  | 0.6280 | 0.7184   | 0.6450  | 0.0734  | 0.882903  | 0.850442 | 0.821187     | C=ent, D=10, L=9, S=15  |
| 5                                          | 5  | 0.6280 | 0.7184   | 0.6450  | 0.0734  | 0.882903  | 0.850442 | 0.821187     | C=ent, D=10, L=9, S=10  |
| 6                                          | 7  | 0.6265 | 0.7136   | 0.6561  | 0.0575  | 0.879844  | 0.857240 | 0.832082     | C=ent, D=10, L=11, S=15 |
| 7                                          | 7  | 0.6265 | 0.7136   | 0.6561  | 0.0575  | 0.879844  | 0.857240 | 0.832082     | C=ent, D=10, L=11, S=10 |
| 8                                          | 7  | 0.6265 | 0.7136   | 0.6561  | 0.0575  | 0.879844  | 0.857240 | 0.832082     | C=ent, D=10, L=11, S=20 |
| 9                                          | 10 | 0.6203 | 0.6804   | 0.6172  | 0.0632  | 0.843474  | 0.804895 | 0.807073     | C=gin, D=10, L=7, S=10  |

--- [모델 비교] 상위 10개 Decision Tree 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|------------------------|
| 0 | 1  | 0.6038 | 0.6294   | 0.5863  | 0.0430  | 0.795377  | 0.765466 | 0.805125     | C=ent, D=8, L=7, S=10  |
| 1 | 2  | 0.6031 | 0.6266   | 0.5856  | 0.0410  | 0.794018  | 0.766145 | 0.804263     | C=ent, D=8, L=7, S=20  |
| 2 | 3  | 0.6024 | 0.6243   | 0.5823  | 0.0420  | 0.791128  | 0.762067 | 0.803444     | C=ent, D=8, L=7, S=25  |
| 3 | 4  | 0.6018 | 0.6281   | 0.5909  | 0.0372  | 0.794697  | 0.769545 | 0.804851     | C=ent, D=8, L=10, S=10 |
| 4 | 4  | 0.6018 | 0.6281   | 0.5909  | 0.0372  | 0.794697  | 0.769545 | 0.804851     | C=ent, D=8, L=10, S=20 |
| 5 | 6  | 0.6010 | 0.6244   | 0.5871  | 0.0374  | 0.790619  | 0.764786 | 0.804196     | C=ent, D=8, L=10, S=25 |
| 6 | 7  | 0.6001 | 0.6232   | 0.5759  | 0.0473  | 0.789599  | 0.753909 | 0.793860     | C=ent, D=8, L=20, S=10 |
| 7 | 7  | 0.6001 | 0.6232   | 0.5759  | 0.0473  | 0.789599  | 0.753909 | 0.793860     | C=ent, D=8, L=20, S=20 |
| 8 | 7  | 0.6001 | 0.6232   | 0.5759  | 0.0473  | 0.789599  | 0.753909 | 0.793860     | C=ent, D=8, L=20, S=25 |
| 9 | 10 | 0.5982 | 0.6211   | 0.5746  | 0.0465  | 0.787220  | 0.753909 | 0.793473     | C=ent, D=8, L=15, S=20 |

--- [모델 비교] 상위 10개 Decision Tree 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                 |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|-------------------------|
| 0 | 1  | 0.6891 | 0.8721   | 0.7026  | 0.1695  | 0.961421  | 0.904827 | 0.819675     | C=ent, D=20, L=7, S=10  |
| 1 | 2  | 0.6762 | 0.8570   | 0.6962  | 0.1608  | 0.955982  | 0.900748 | 0.822006     | C=ent, D=20, L=7, S=20  |
| 2 | 3  | 0.6730 | 0.8422   | 0.6911  | 0.1511  | 0.950374  | 0.897349 | 0.825200     | C=ent, D=20, L=7, S=25  |
| 3 | 4  | 0.6672 | 0.7980   | 0.6790  | 0.1189  | 0.929130  | 0.880354 | 0.817280     | C=gin, D=20, L=10, S=10 |
| 4 | 4  | 0.6672 | 0.7980   | 0.6790  | 0.1189  | 0.929130  | 0.880354 | 0.817280     | C=gin, D=20, L=10, S=20 |
| 5 | 6  | 0.6639 | 0.7753   | 0.6738  | 0.1015  | 0.918423  | 0.874915 | 0.825984     | C=gin, D=20, L=10, S=25 |
| 6 | 7  | 0.6630 | 0.8218   | 0.6721  | 0.1497  | 0.938817  | 0.873555 | 0.809099     | C=gin, D=20, L=7, S=10  |
| 7 | 8  | 0.6610 | 0.8508   | 0.7003  | 0.1505  | 0.953433  | 0.900748 | 0.825347     | C=ent, D=20, L=10, S=10 |
| 8 | 8  | 0.6610 | 0.8508   | 0.7003  | 0.1505  | 0.953433  | 0.900748 | 0.825347     | C=ent, D=20, L=10, S=20 |
| 9 | 10 | 0.6602 | 0.8379   | 0.6931  | 0.1447  | 0.948165  | 0.897349 | 0.837179     | C=ent, D=20, L=10, S=25 |

튜닝을 진행해도, SVM의 결과를 따라잡기는 힘들었다.

이 결과는 튜닝이 덜 되어서라기보다는 단일 트리 모델이 가지는 수학적, 구조적인 근본적 한계 때문이다.

이전 리포트의 EDA와 SVM 결정경계 시각화에서 봤듯이, 이 데이터의 고장과 정상은 대각선이나 곡선 형태의 매우 복잡하고 둥근 비선형 경계로 얹혀있다. 단일 트리가 수직,수평선만 그어서는 이 복잡한 대각선과 곡선 경계를 절대 부드럽게 나눌 수 없기 때문이다. 따라서 SVM이 RBF 커널로 데이터를 고차원으로 구부려 성능을 높인 것과 대조되는 것이다.

모델의 순위들을 보면 **gini**가 아니라 **entropy**로 측정된 모델들이 상위권에 있는데 이는 클래스 불균형이 극심하고 **entropy**는 로그함수를 쓰기에 소수 클래스에 더 민감하게 반응하여 트리를 조금 더 세밀하고 정교하게 가지치기 했기에 좀 더 높은 점수에 도달했다.

```
[선택된 2위 모델의 하이퍼파라미터]
• criterion: entropy
• max_depth: 15
• min_samples_leaf: 7
• min_samples_split: 5

[Train 데이터 평가]
Accuracy (정확도) : 0.9514
F1-Macro Score    : 0.849432
ROC-AUC Score     : 0.988685

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.99      | 0.95   | 0.97     | 5470    |
| 1 (Failure)  | 0.60      | 0.91   | 0.73     | 414     |
| accuracy     |           |        | 0.95     | 5884    |
| macro avg    | 0.80      | 0.93   | 0.85     | 5884    |
| weighted avg | 0.97      | 0.95   | 0.96     | 5884    |

```

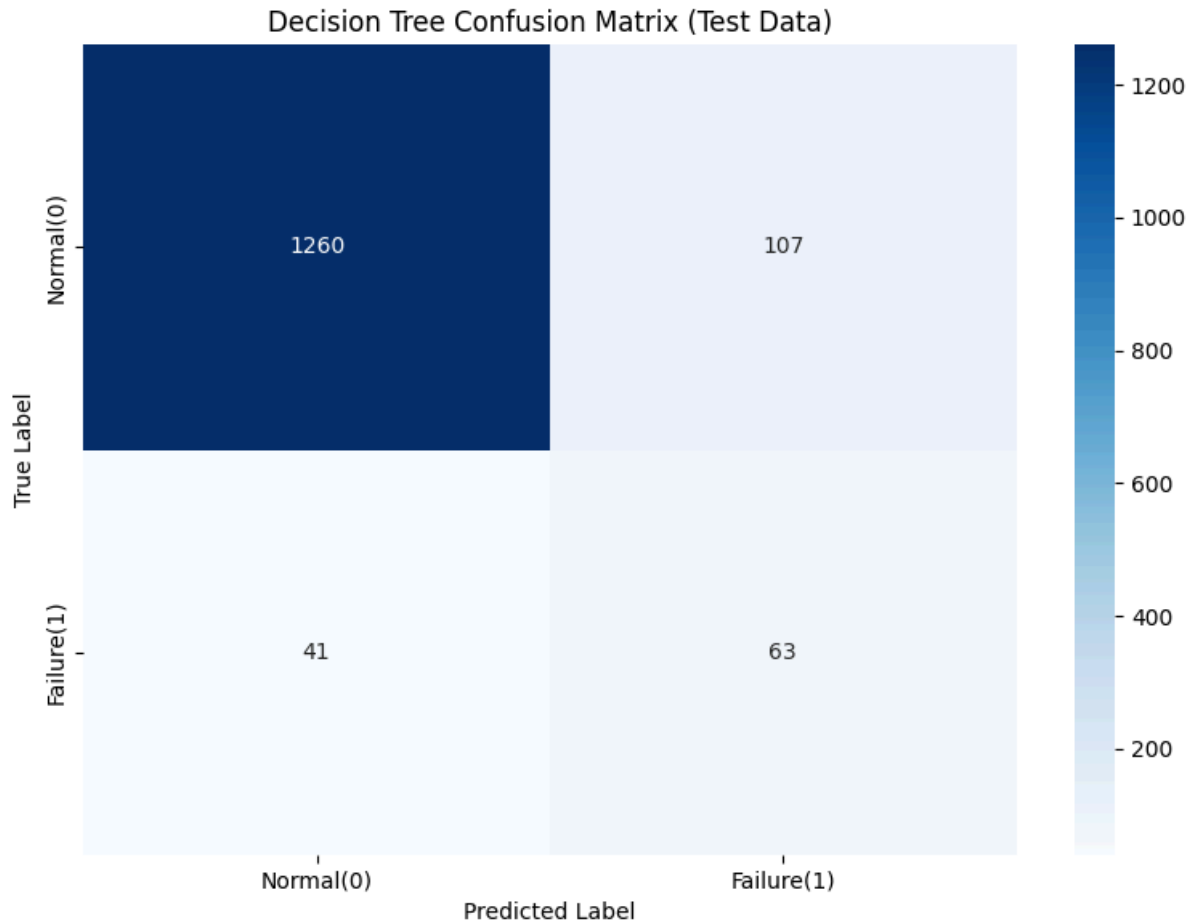

[Test 데이터 평가]
Accuracy (정확도) : 0.9001
F1-Macro Score    : 0.707077
ROC-AUC Score     : 0.821581

[Test 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.92   | 0.94     | 1367    |
| 1 (Failure)  | 0.38      | 0.62   | 0.47     | 104     |
| accuracy     |           |        | 0.90     | 1471    |
| macro avg    | 0.67      | 0.77   | 0.71     | 1471    |
| weighted avg | 0.93      | 0.90   | 0.91     | 1471    |

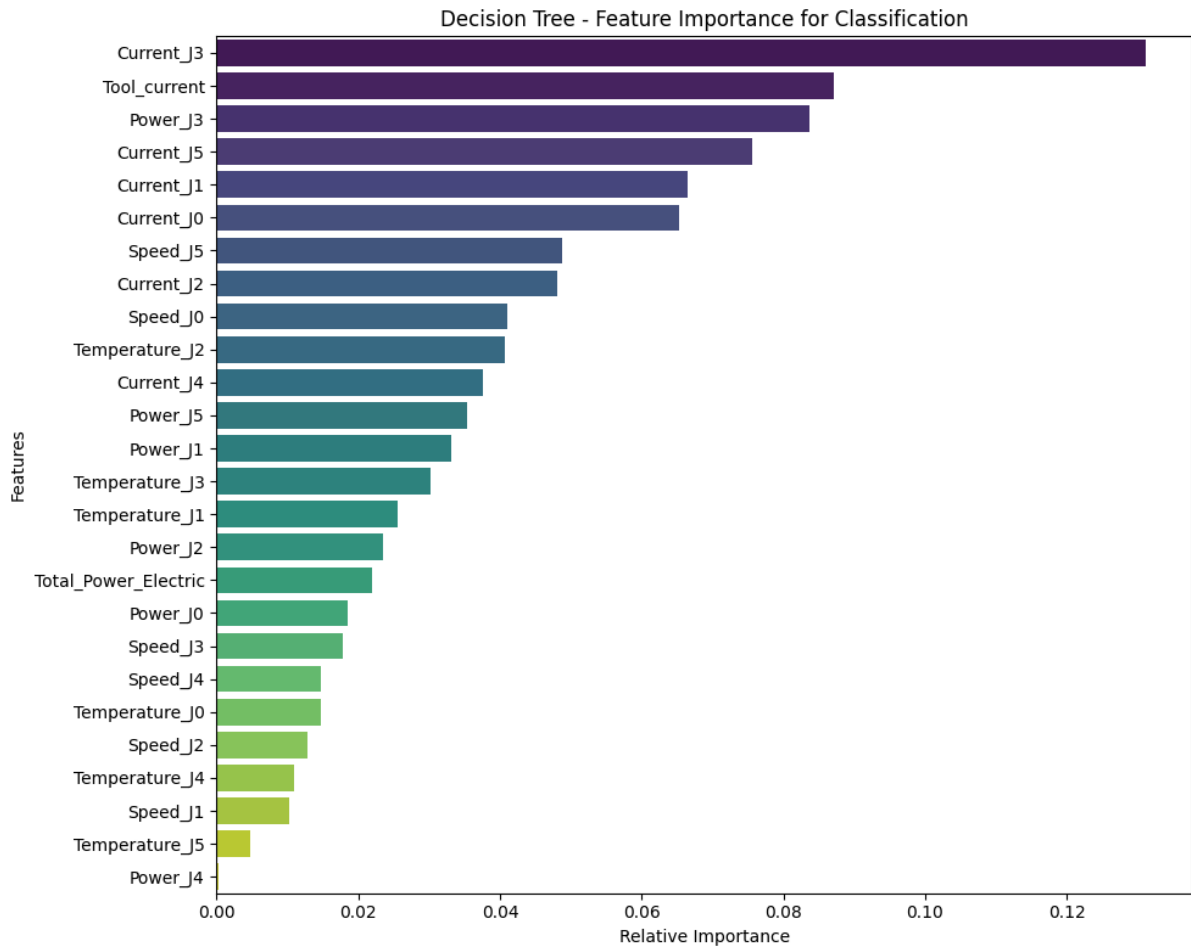
가장 성능이 좋았던 처음의 2위 모델의 세부 지표를 보면, **Train**에서는 지표들이 굉장히 좋지만 **Test**에서는 SVM보다 ROC-AUC, F1, Precision, Recall 모두 밀린다. 이는 모델이 **Train** 데이터를 전부 암기했기 때문에 **Test** 데이터를 보자 과적합을 드러낸것이다.



오차행렬에서는 DT가 FP를 107건으로, SVM이 117건으로 10번 덜 오해했지만,

TP는 DT가 63건으로, SVM이 75건으로 SVM이 더 높다.

트리는 데이터를 자를 때 대각선이나 곡선이 아닌, 오직 수직,수평으로만 자른다. 고장 데이터와 정상 데이터가 복잡하게 겹쳐있는 영역에서, DT는 이를 사각형 블록 모양으로 억지로 감싸게 된다. 이 사각형 모서리 바깥으로 살짝만 빗겨 나간 실제 고장 데이터들은 트리가 포착하지 못하고 정상으로 놓치게 되며 (FN 41건), 반대로 사각형 안쪽에 우연히 섞여 있던 정상 데이터들은 사각형에 갇혀 고장으로 오판(FP 107건)받게 된 것이다. 즉, 유연한 곡선으로 데이터를 감싸는 SVM과 달리, 딱딱한 블록 형태로 경계를 긋는 트리의 한계 때문에 진짜 고장을 더 많이 놓치게 되었다.



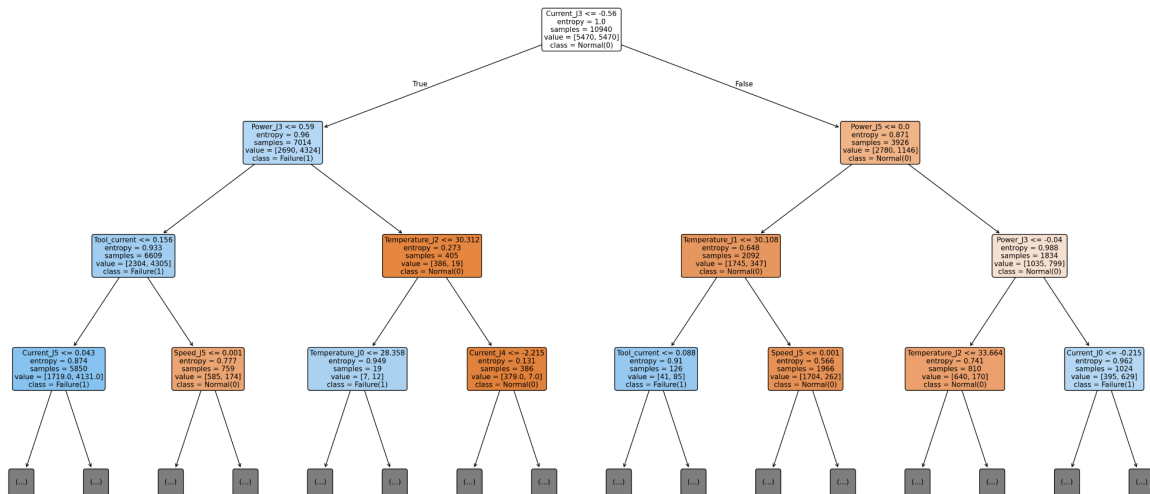
피쳐 중요도 산출 결과, LR에서의 피쳐 중요도가 공통적으로 Temperature J3을 비롯한 온도와 속도를 중요하게 판단했다면, DT는 다르게 Current J3, Tool current, Power J3 등 전류와 관련된 변수들을 중요하게 판단했다.

LR은 전체적인 1차원 비례관계를 찾는다면 트리는 분기점마다 정상과 고장을 가장 확실하게 가를 수 있는 단 하나의 변수와 기준값을 찾는다. 로봇팔에 비정상 작동이 발생하는 순간을 상상해 보면, 온도나 속도는 서서히 변하지만 전류는 물리적인 충돌이나 비정상적인 힘이 가해질 때 순간적으로 확 튀어 오르는 특성이 있다. 트리는 이 순간적인 전류 값을 고장을 가르는 기준선으로 판단했을 것이다.

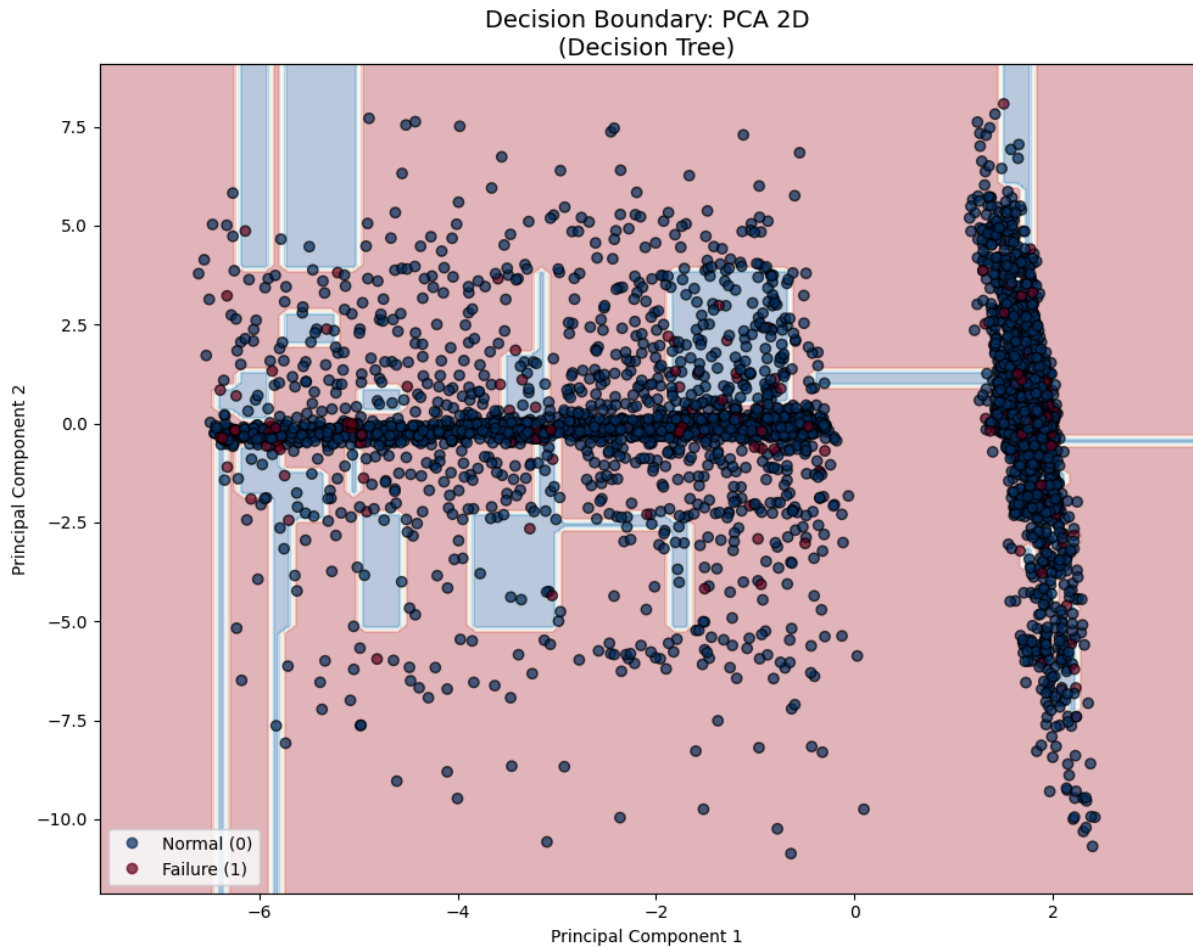
물리적인 상식을 DT가 잡아낸것으로 점수는 낮을지언정 제대로된 기준을 가지고 판단하기 시작했다는 좋은 성과이다.



### Decision Tree Structure Analysis (Top 3 Levels)



모델의 의사결정과정을 보면, 가장 첫번째로 **Current J3**이 **-0.56**(Standard Scaling된 값)보다 작은지 큰지 판단한다. 여기서 **SMOTE**로 생성된 데이터들과 원래 데이터들이 **1:1**로 나누어진다. 만약 **-0.56**보다 작다면 고장일 확률이 크게 상승한다. 왼쪽 가지들을 보면 그 다음으로 **Power J3**가 낮고, **Tool current**도 낮다면 매우 강력하게 고장 상태로 확신한다. 만약, **Level 1**의 **Power J3**에서 **False**라면 박스의 색이 진한 주황색으로 바뀐다. 즉, 조건 하나가 바뀌자 고장 위험에서 정상 상태로 분류되어버린다. 트리 기반 모델이 복잡한 비선형적 판단을 해내는것을 보여준다.



Decision Boundary를 보면 SVM처럼 부드럽고 둥근 곡선이 아닌 수직, 수평으로 깎인 사각형 모양이다. 대각선이나 곡선으로 뭉쳐 있는 데이터를 부드럽게 감싸지 못하고 억지로 네모 반듯하게 자르다가 놓침을 많이 발생시킨것 같다.

단일 Decision Tree 모델을 통해, 로봇 고장의 물리적 원인인 전류와 파워 변수의 중요성을 성공적으로 검증해 냈다. 그러나 단일 트리는 필연적으로 심각한 과적합을 발생시켰으며, 수직/수평으로만 꺾이는 계단식 경계선 때문에 데이터의 곡선 분포를 부드럽게 감싸지 못해 재현율이 하락했다. 이를 해결하기 위해, 수백 개의 트리를 앙상블하여 과적합을 완벽하게 방지하고, SVM처럼 부드러운 비선형 결정 경계를 만들어 낼 수 있는 Random Forest 모델을 도입하여 성능의 정점을 확인하고자 한다.

## 6. Random Forest

여러 트리를 모아 판단하는 앙상블 모델인 Random Forest로 학습을 진행해본다.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"학습 데이터 크기: {X_train_scaled.shape}")
print(f"테스트 데이터 크기: {X_test_scaled.shape}")
```

✓ 0.0s

학습 데이터 크기: (5884, 26)  
테스트 데이터 크기: (1471, 26)

RF는 Standard Scaling을 진행할 필요는 없지만, 클래스 불균형을 해소하기 위해 거리

기반인 SMOTE가 동작하기 때문에 일관성을 위해 Standard Scaling을 진행한다.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축 (SMOTE -> RandomForest)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('rf', RandomForestClassifier(random_state=42))
])

# 파라미터 그리드
param_grid = {
    'rf_n_estimators': [100, 200],          # 트리의 개수 (앙상블 규모)
    'rf_max_depth': [10, 15, 20, None],     # 트리의 최대 깊이
    'rf_min_samples_split': [2, 5, 10],     # 노드를 분할하기 위한 최소 샘플 수
    'rf_min_samples_leaf': [1, 2, 4]       # 리프 노드가 되기 위한 최소 샘플 수
}

print("--- Random Forest 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"  • {param.replace('rf_', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

랜덤포레스트 학습과 Grid SearchCV를 진행해본다. 하이퍼파라미터로는 트리의 갯수와 DT에서 사용했던 최대 깊이, 노드를 분할하기 위한 최소 샘플 수, 리프 노드가 되기 위한 최소 샘플 수가 있다.

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                  |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|--------------------------|
| 0 | 1  | 0.7560 | 0.9904   | 0.7964  | 0.1939  | 0.997451  | 0.946975 | 0.935242     | E=100, D=None, S=2, L=2  |
| 1 | 2  | 0.7551 | 0.9898   | 0.7876  | 0.2021  | 0.997281  | 0.944935 | 0.937380     | E=200, D=None, S=2, L=2  |
| 2 | 3  | 0.7548 | 0.9885   | 0.7791  | 0.2094  | 0.996941  | 0.942216 | 0.942856     | E=200, D=None, S=5, L=2  |
| 3 | 4  | 0.7536 | 0.9564   | 0.7954  | 0.1611  | 0.987933  | 0.942896 | 0.935675     | E=200, D=None, S=10, L=4 |
| 4 | 5  | 0.7535 | 1.0000   | 0.7891  | 0.2109  | 1.000000  | 0.946295 | 0.941365     | E=200, D=None, S=2, L=1  |
| 5 | 6  | 0.7530 | 0.9987   | 0.7910  | 0.2077  | 0.999660  | 0.946295 | 0.938471     | E=200, D=None, S=5, L=1  |
| 6 | 7  | 0.7527 | 0.9658   | 0.7935  | 0.1722  | 0.990483  | 0.943576 | 0.930522     | E=100, D=20, S=5, L=1    |
| 7 | 8  | 0.7527 | 0.9879   | 0.7827  | 0.2051  | 0.996771  | 0.942896 | 0.936684     | E=100, D=None, S=5, L=2  |
| 8 | 9  | 0.7523 | 0.9594   | 0.7937  | 0.1657  | 0.988783  | 0.942896 | 0.935210     | E=100, D=None, S=10, L=4 |
| 9 | 10 | 0.7520 | 0.9841   | 0.7879  | 0.1962  | 0.995751  | 0.944256 | 0.929854     | E=100, D=None, S=10, L=1 |

Test F1, Accuracy, ROC-AUC 모든 면에서 가장 성능이 좋았던 SVM보다 더 높은 수치를 기록했다. 하지만 Train 데이터를 거의 그대로 암기한 탓에 조금 과적합되어 과적합 Gap이 평균적으로 0.19정도를 기록했다.

```

[선택된 4위 모델의 하이퍼파라미터]
• max_depth: None
• min_samples_leaf: 4
• min_samples_split: 10
• n_estimators: 200

[Train 데이터 평가]
Accuracy (정확도) : 0.9879
F1-Macro Score    : 0.956444
ROC-AUC Score     : 0.999100

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 1.00      | 0.99   | 0.99     | 5470    |
| 1 (Failure)  | 0.87      | 0.98   | 0.92     | 414     |
| accuracy     |           |        | 0.99     | 5884    |
| macro avg    | 0.93      | 0.98   | 0.96     | 5884    |
| weighted avg | 0.99      | 0.99   | 0.99     | 5884    |

```

[Test 데이터 평가]
Accuracy (정확도) : 0.9429
F1-Macro Score    : 0.795370
ROC-AUC Score     : 0.935675

[Test 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.96   | 0.97     | 1367    |
| 1 (Failure)  | 0.58      | 0.66   | 0.62     | 104     |
| accuracy     |           |        | 0.94     | 1471    |
| macro avg    | 0.78      | 0.81   | 0.80     | 1471    |
| weighted avg | 0.95      | 0.94   | 0.94     | 1471    |

가장 과적합 gap과 F1간의 균형이 좋은 4위 모델의 세부 지표를 보면 Precision이 0.58,

Recall은 0.66, F1 Score가 0.62, ROC-AUC가 0.93을 기록했다.

전까지의 가장 성능이 좋았던 SVM의 SMOTE 언더샘플링과 비교해보면 Precision은 0.18이

올랐지만 Recall은 0.7정도가 감소했다. F1은 0.1이 올랐다.

```
# 파라미터 그리드
param_grid = {
    # 'rf__n_estimators': [100, 200],           # 트리의 개수 (앙상블 규모)
    # 'rf__max_depth': [7, 8, 10],             # 트리의 최대 깊이
    # 'rf__min_samples_split': [8, 12, 14],     # 노드를 분할하기 위한 최소 샘플 수
    # 'rf__min_samples_leaf': [15, 20, 25, 30]  # 리프 노드가 되기 위한 최소 샘플 수
    'rf__n_estimators': [100, 200],           # 숲의 크기 (트리가 많을수록 안정적, 과적합을 유발하지 않음)
    'rf__max_depth': [15, 20, 25],           # 트리의 최대 깊이 제한
    'rf__min_samples_split': [8, 10, 15],    # 노드를 쪼개기 위한 최소 샘플 수 (허들 높이기)
    'rf__min_samples_leaf': [2, 4, 6],       # 이파리(끝 노드)에 남아야 하는 최소 샘플 수 (잔가지 치기)
    'rf__max_features': ['sqrt', 0.5, 0.7]   # 분할 시 고려할 최대 변수 개수 (트리들의 다양성 확보)
}
```

여러번 하이퍼파라미터 튜닝을 시도해보았다.

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                        |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|--------------------------------|
| 0 | 1  | 0.7527 | 0.9732   | 0.8001  | 0.1731  | 0.992692  | 0.946295 | 0.935038     | E=100, D=25, S=8, L=2, F=sqrt  |
| 1 | 2  | 0.7518 | 0.9756   | 0.7983  | 0.1773  | 0.993372  | 0.946295 | 0.937919     | E=200, D=25, S=8, L=2, F=sqrt  |
| 2 | 3  | 0.7518 | 0.9544   | 0.7919  | 0.1624  | 0.987254  | 0.942896 | 0.934050     | E=100, D=25, S=10, L=4, F=sqrt |
| 3 | 4  | 0.7503 | 0.9690   | 0.7984  | 0.1705  | 0.991502  | 0.945615 | 0.936800     | E=200, D=25, S=10, L=2, F=sqrt |
| 4 | 5  | 0.7499 | 0.9514   | 0.7970  | 0.1544  | 0.986404  | 0.943576 | 0.935724     | E=200, D=25, S=10, L=4, F=sqrt |
| 5 | 6  | 0.7497 | 0.9525   | 0.7937  | 0.1589  | 0.986574  | 0.942896 | 0.937384     | E=200, D=20, S=8, L=2, F=sqrt  |
| 6 | 7  | 0.7491 | 0.9342   | 0.7891  | 0.1451  | 0.981135  | 0.940177 | 0.937468     | E=100, D=25, S=10, L=6, F=sqrt |
| 7 | 7  | 0.7491 | 0.9342   | 0.7891  | 0.1451  | 0.981135  | 0.940177 | 0.937468     | E=100, D=25, S=8, L=6, F=sqrt  |
| 8 | 9  | 0.7486 | 0.9264   | 0.7876  | 0.1388  | 0.978756  | 0.939497 | 0.940788     | E=100, D=25, S=15, L=6, F=sqrt |
| 9 | 10 | 0.7481 | 0.9584   | 0.8018  | 0.1566  | 0.988443  | 0.946295 | 0.938643     | E=200, D=25, S=8, L=4, F=sqrt  |

튜닝 결과 10순위 모델의 Test F1이 0.8018으로 늘었고, 과적합 Gap이 0.1566으로 줄었다.

Train F1의 점수가 낮아져 억지로 다 외우지 않게되며 F1 점수도 올라가게되어 Gap이 줄어들었다.

```

[선택된 10위 모델의 하이퍼파라미터]
• max_depth: 25
• max_features: sqrt
• min_samples_leaf: 4
• min_samples_split: 8
• n_estimators: 200

[Train 데이터 평가]
Accuracy (정확도) : 0.9884
F1-Macro Score    : 0.958415
ROC-AUC Score     : 0.999236

[Train 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 1.00      | 0.99   | 0.99     | 5470    |
| 1 (Failure)  | 0.87      | 0.99   | 0.92     | 414     |
| accuracy     |           |        | 0.99     | 5884    |
| macro avg    | 0.93      | 0.99   | 0.96     | 5884    |
| weighted avg | 0.99      | 0.99   | 0.99     | 5884    |

```

[Test 데이터 평가]
Accuracy (정확도) : 0.9463
F1-Macro Score    : 0.801794
ROC-AUC Score     : 0.938643

[Test 상세 분류 리포트]

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.97   | 0.97     | 1367    |
| 1 (Failure)  | 0.61      | 0.65   | 0.63     | 104     |
| accuracy     |           |        | 0.95     | 1471    |
| macro avg    | 0.79      | 0.81   | 0.80     | 1471    |
| weighted avg | 0.95      | 0.95   | 0.95     | 1471    |

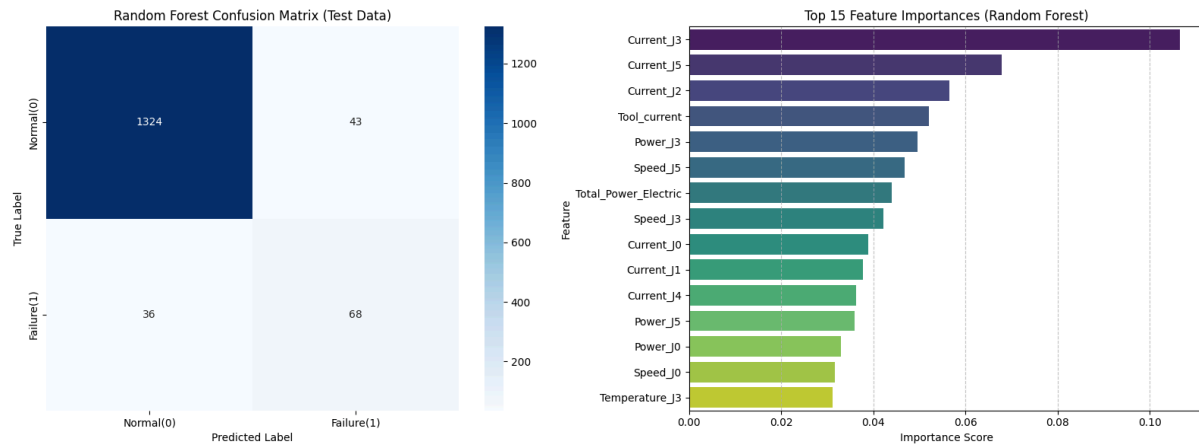
Precision은 0.61로, 모델이 비정상이라고 예측한 데이터 중 약 61%가 실제 비정상 데이터이다.

Recall은 0.65로, 실제 비정상 데이터의 약 65%를 모델이 성공적으로 탐지한다.

F1 score는 0.63으로, Precision과 Recall을 종합적으로 고려한 성능을 나타낸다.

ROC-AUC는 0.93, 정상과 비정상을 구별하는 분류 능력이 93%로 뛰어나다.





오차행렬을 그려본 결과,

TN은 1324건

FP은 43건

FN은 36건

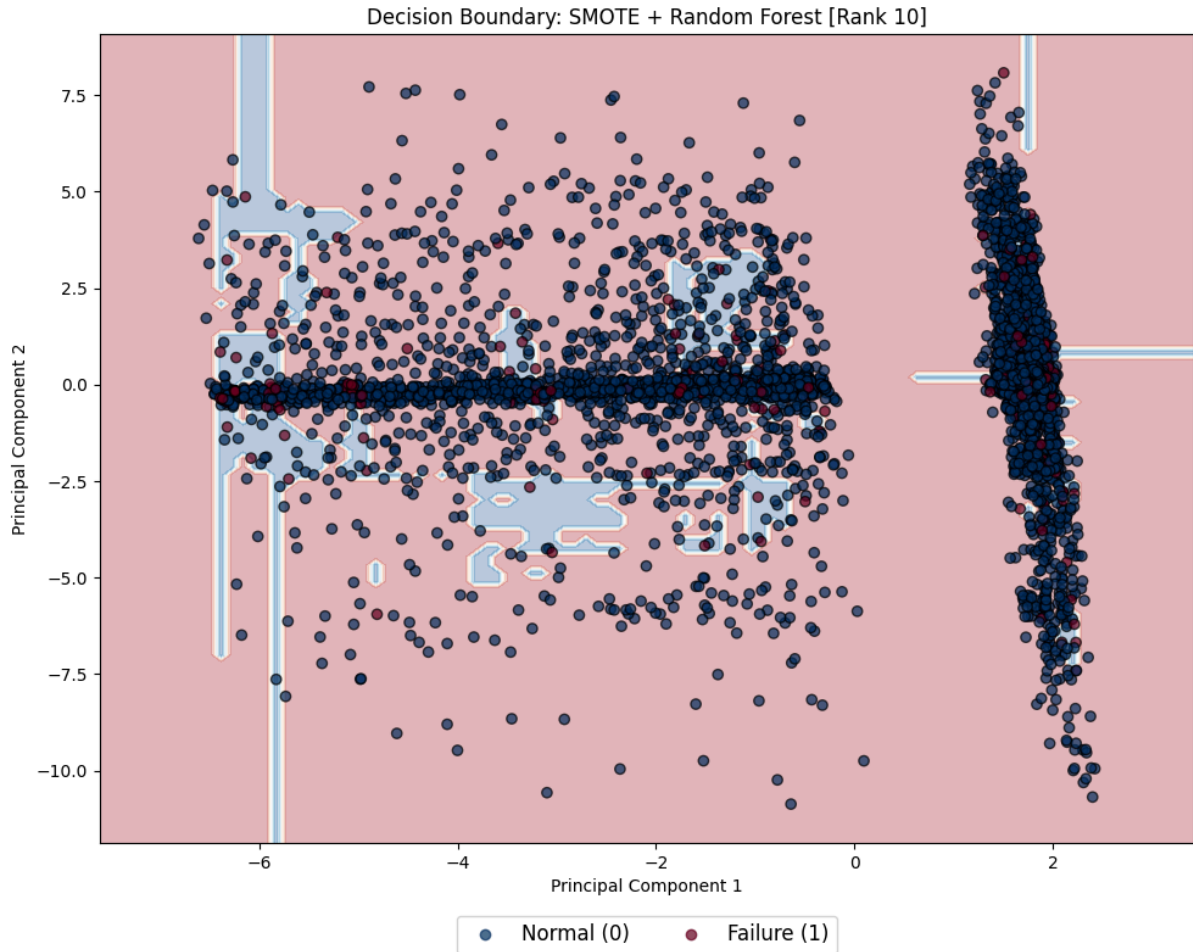
TP은 68건

으로 처음의 LR에서 FP는 480건, SVM에서 115건, RF에서 43건을 기록해 앞서 추측한대로 앙상블 기반 트리모델인 RF에서 가장 뛰어난 성능을 보이고있다.

Feature Importance 분석 결과 Current J3, Current J5, Current J2가 가장 높은 중요도를 보였다.

특히 상위 중요 변수 대부분이 전류 관련 변수로 나타나, 장비의 이상 여부를 판단하는 데 전류 변화가 핵심적인 역할을 하는 것으로 해석된다.

또한 Power J3, Total Power Electric 등의 Current를 사용해 만든 파생 변수 역시 높은 중요도를 보여 전류 및 전력 계열 특성이 고장 예측에 중요한 영향을 미치는 것으로 확인되었다.



Decision boundary 분석 결과 모델의 경계선이 직각 형태의 계단식으로 형성되어 있다. 이는 연속된 실수 데이터를 특정 임계값 기준으로 수직,수평 분할하는 트리 기반 알고리즘 고유의 특성이다.

수백 개의 트리가 형성한 직각의 경계선들이 중첩되어, 데이터의 복잡한 굴곡을 세밀하게 반영하는 모자이크 형태의 고도화된 결정 경계를 완성하였다.

SMOTE로 인해 생성된 경계선 주변의 데이터들을 언더샘플링을 실행하여 지워보았다.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축 (SMOTE -> ENN -> RandomForest)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('enn', EditedNearestNeighbours(n_neighbors=3, kind_sel='all')),
    ('rf', RandomForestClassifier(random_state=42))
])

# 파라미터 그리드
param_grid = {
    # 'rf__n_estimators': [100, 200],
    # 'rf__max_depth': [10, 15, 20, None],
    # 'rf__min_samples_split': [2, 5, 10],
    # 'rf__min_samples_leaf': [1, 2, 4]
    'rf__n_estimators': [100, 200], # 숲의 크기 (트리가 많을수록 안정적, 과적합을 유발하지 않음)
    'rf__max_depth': [15, 20, 25], # 트리의 최대 깊이 제한
    'rf__min_samples_split': [8, 10, 15], # 노드를 쪼개기 위한 최소 샘플 수 (허를 높이기)
    'rf__min_samples_leaf': [2, 4, 6], # 이파리(끝 노드)에 남아야 하는 최소 샘플 수 (잔가지 치기)
    'rf__max_features': ['sqrt'] # 분할 시 고려할 최대 변수 개수 (트리들의 다양성 확보)
}

print("--- [SMOTE+ENN] Random Forest 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f"    • {param.replace('rf__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

SMOTE를 진행한 다음, 언더샘플링을 통해 경계선 주변 데이터들을 없애고 RF를 진행한다.

하이퍼파라미터는 최고성능이 나왔던 수치로 진행한다.

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정                        |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|--------------------------------|
| 0 | 1  | 0.7549 | 0.9696   | 0.7999  | 0.1697  | 0.991842  | 0.947655 | 0.936441     | E=200, D=25, S=8, L=2, F=sqrt  |
| 1 | 2  | 0.7541 | 0.9684   | 0.7931  | 0.1753  | 0.991502  | 0.945615 | 0.937004     | E=100, D=25, S=8, L=2, F=sqrt  |
| 2 | 3  | 0.7504 | 0.9526   | 0.8018  | 0.1508  | 0.986914  | 0.946295 | 0.941267     | E=100, D=25, S=8, L=4, F=sqrt  |
| 3 | 4  | 0.7497 | 0.9654   | 0.8000  | 0.1654  | 0.990653  | 0.946975 | 0.938010     | E=200, D=25, S=10, L=2, F=sqrt |
| 4 | 5  | 0.7497 | 0.9401   | 0.7937  | 0.1465  | 0.983175  | 0.942896 | 0.938917     | E=200, D=20, S=10, L=4, F=sqrt |
| 5 | 6  | 0.7495 | 0.9383   | 0.7903  | 0.1479  | 0.982665  | 0.942216 | 0.939670     | E=200, D=25, S=10, L=6, F=sqrt |
| 6 | 6  | 0.7495 | 0.9383   | 0.7903  | 0.1479  | 0.982665  | 0.942216 | 0.939670     | E=200, D=25, S=8, L=6, F=sqrt  |
| 7 | 8  | 0.7490 | 0.9583   | 0.7865  | 0.1718  | 0.988613  | 0.942896 | 0.937271     | E=200, D=25, S=8, L=4, F=sqrt  |
| 8 | 9  | 0.7480 | 0.9368   | 0.7889  | 0.1478  | 0.982155  | 0.940857 | 0.936828     | E=100, D=25, S=8, L=6, F=sqrt  |
| 9 | 9  | 0.7480 | 0.9368   | 0.7889  | 0.1478  | 0.982155  | 0.940857 | 0.936828     | E=100, D=25, S=10, L=6, F=sqrt |

두 방법 모두 최고 Test F1이 0.8018로 유사한 점수를 보였다.

앞서 진행한 LR,SVM 같은 극적인 향상은 나타나지 않았다.

하지만 최고 모델인 3순위 모델을 보면, 언더샘플링을 진행하기 전보다 ROC-AUC가

0.003정도 미세하게 상승했고 과적합 Gap도 0.006정도로 미세하게 감소했다.

따라서 데이터의 노이즈 제거 효과로 인해 일반화 성능이 일부 개선된 것으로 판단된다.

```
[선택된 3위 모델의 하이퍼파라미터]
• max_depth: 25
• max_features: sqrt
• min_samples_leaf: 4
• min_samples_split: 8
• n_estimators: 100

[Train 데이터 평가]
Accuracy (정확도) : 0.9869
F1-Macro Score   : 0.952565
ROC-AUC Score    : 0.998147

[Train 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 1.00 | 0.99 | 0.99 | 5470 |
| 1 (Failure) | 0.86 | 0.97 | 0.91 | 414 |
| | | | | |
| | accuracy | | | 0.99 | 5884 |
| | macro avg | 0.93 | 0.98 | 0.95 | 5884 |
| | weighted avg | 0.99 | 0.99 | 0.99 | 5884 |

[Test 데이터 평가]
Accuracy (정확도) : 0.9463
F1-Macro Score   : 0.801794
ROC-AUC Score    : 0.941267

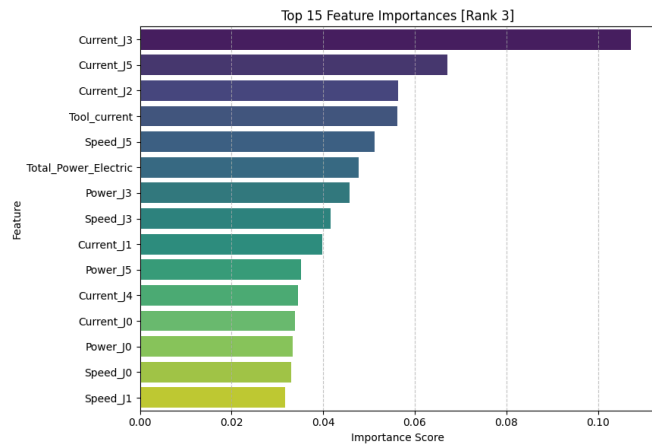
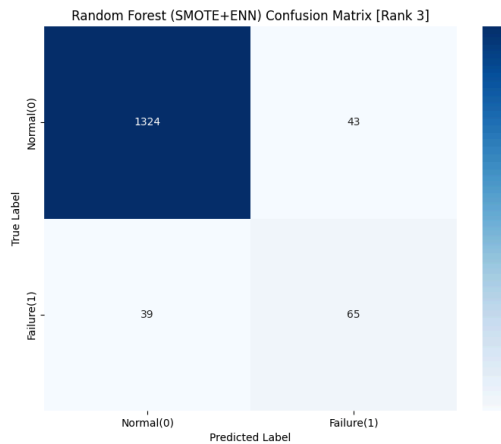
[Test 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 0.97 | 0.97 | 0.97 | 1367 |
| 1 (Failure) | 0.61 | 0.65 | 0.63 | 104 |
| | | | | |
| | accuracy | | | 0.95 | 1471 |
| | macro avg | 0.79 | 0.81 | 0.80 | 1471 |
| | weighted avg | 0.95 | 0.95 | 0.95 | 1471
```

학습 데이터의 노이즈를 제거하여 과적합 정도를 소폭 감소시켰으나, 테스트 데이터의

Accuracy, F1 Score, Precision, Recall은 SMOTE 적용 결과와 거의 동일하게 나타났다.

따라서 RF에서는 SMOTE에 언더샘플링을 더한것이 일반화 성능을 크게 향상시키지는

못했지만, 보다 안정적인 모델 학습에 기여한 것으로 판단된다.



TP = 65

FN = 39

FP = 43

TN = 1324

로 기존의 **SMOTE**만 적용한 모델의 오차행렬보다 **TP**가 3감소하고 **FN**이 3증가한 정도로 거의 유사하다.

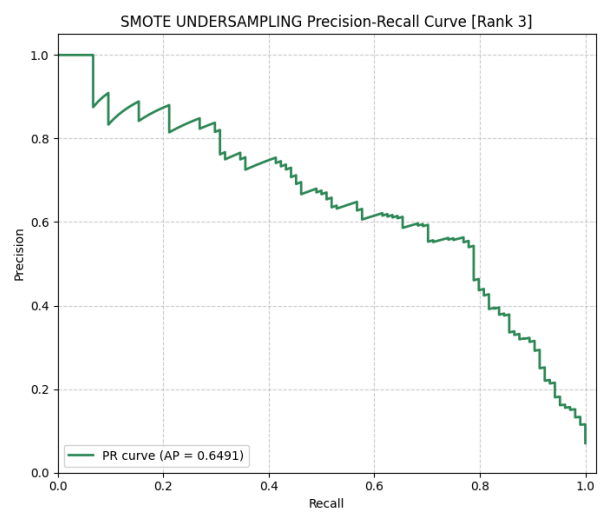
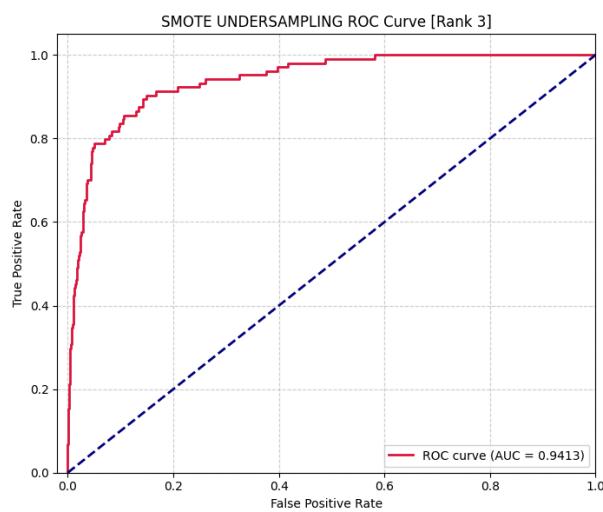
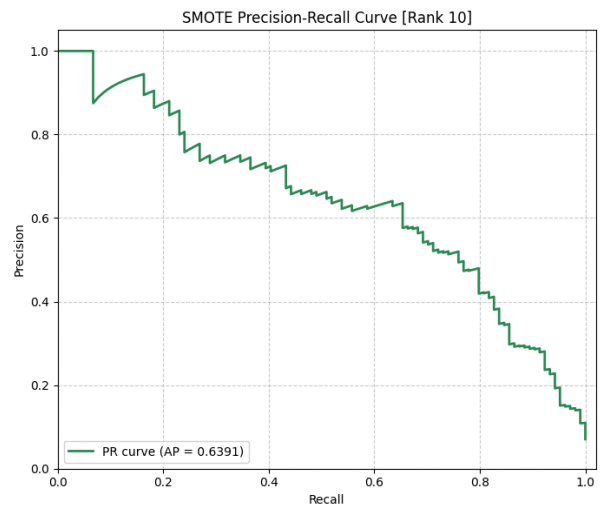
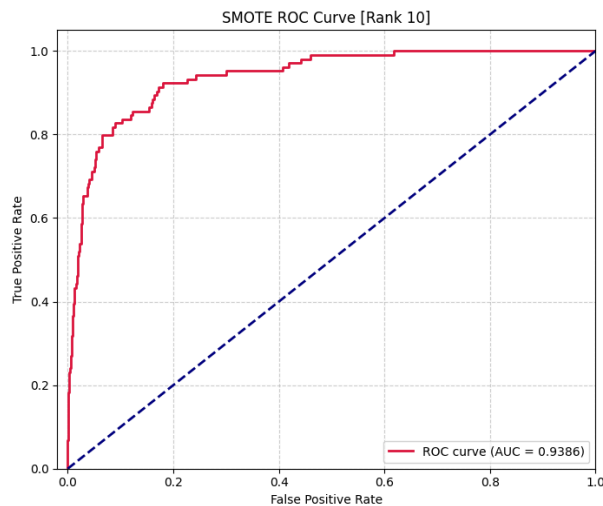
**Feature Importance**도 4위까지 같고 **Power J3**와 **Total Power Electric**, **Speed J5**의 순위가 조금 바뀐 정도로 핵심은 동일하다.

이 의미는 **SMOTE**만으로도 **RF**는 이미 데이터의 핵심 구조를 잘 학습하고 있다는걸 나타낸다.

트리기반 앙상블 모델인 **RF**는 몇백개의 나무가 다수결 투표를 한다. 데이터에 노이즈가 섞여서 나무 10그루가 오답을 내도 나머지 190그루의 나무가 노이즈가 전체 결과에 미치는 영향을 알아서 무시한다.

즉, **RF**는 모델 자체 내부에 이미 강력한 노이즈 무시 기능이 있기때문에 언더샘플링으로 데이터를 한번 더 청소해도 점수가 크게 오르지 않았던것으로 판단된다.

하지만 **ROC-AUC**는 0.9412로 현재 최고 점수를 도달했고 과적합 **gap**도 줄어들어 모델이 더 안정되었음으로 아예 의미가 없던것은 아니라고 볼 수 있다.



SMOTE와 SMOTE에 언더샘플링한 모델 각각의 ROC Curve, PR Curve를 그려보았다.

왼쪽의 ROC Curve는 가짜 알람(False Positive Rate)을 통제하면서 실제 로봇 고장(True Positive Rate, Recall)을 얼마나 잘 감지하는지를 나타내는 지표이다. 곡선이 좌측 상단에

밀착할수록 정상과 고장을 명확하게 구분하는 우수한 모델임을 의미한다.

두 모델 각각 ROC-AUC가 0.9386, 0.9413 이므로 매우 근접한 탁월한 분류 성능을

입증하였고 격차는 매우 낮다.

정상 데이터가 고장 데이터보다 압도적으로 많은 불균형 데이터셋에서는 ROC Curve가

다소 과대평가되는 경향이 있다. 이를 보완하기 위해 분석하는 우측의 PR Curve는, 실제

고장을 찾아내는 비율(**Recall**)을 높이려 할 때 알람의 신뢰도(**Precision**)가 어떻게 변화하는지를 평가한다.

두 모델 각각 **PR-AP** 점수가 **0.6319, 0.6419**를 기록하였다. 일반적인 불균형 데이터셋에서는 **AP** 점수가 급격하게 **0**으로 추락하는 현상과 달리, 본 모델들의 곡선은 완만한 기울기로 우측을 향해 뻗어간다. 이는 **Recall**을 **60%** 이상 끌어올리는 과정에서도 **Precision**을 **60%** 이상으로 굳건하게 유지하는, 방어력이 높은 모델임을 증명한다.

**Random Forest**는 여러 개의 **Decision Tree**를 앙상블하여 예측을 수행하는 모델로, 데이터의 복잡한 비선형 관계를 학습할 수 있다는 장점을 가진다. 본 데이터셋은 **EDA** 단계에서 **System Failure**와 각 피쳐 간의 선형 상관관계가 매우 약하게 나타났으며, 정상 데이터와 고장 데이터의 분포가 상당 부분 중첩되어 있었다. 따라서 선형 모델보다는 비선형 패턴을 학습할 수 있는 **Random Forest**가 더욱 적합할 것으로 예상하였다.

**SMOTE**를 적용한 **Random Forest**의 최종 모델은 **Test Accuracy 94.63%**, **Macro F1 Score 0.8018**, **ROC-AUC 0.9386**을 기록하였다. 특히 **Failure** 클래스의 **Precision**은 **0.61**, **Recall**은 **0.65**, **F1-score**는 **0.63**으로 나타났다. 이는 실제 고장 데이터의 약 **65%**를 탐지하면서도 예측한 고장 중 약 **61%**가 실제 고장임을 의미한다.

혼동행렬 분석 결과 실제 고장 **104**건 중 **68**건을 정확히 탐지하였으며, **36**건은 정상으로 잘못 분류하였다. 반면 정상 데이터 **1367**건 중 **1324**건을 정확히 분류하여 높은 안정성을 보였다. 이는 이전에 실험한 **Logistic Regression**이나 **SVM**보다 우수한 성능으로, 데이터 내 복잡한 비선형 패턴을 효과적으로 학습했음을 보여준다.

**Feature Importance** 분석 결과 **Current J3**, **Current J5**, **Current J2**, **Tool\_current**, **Power\_J3** 등이 가장 중요한 변수로 나타났다. 특히 상위 중요 변수 대부분이 전류(**Current**) 및

전력(Power) 관련 변수로 구성되어 있었으며, 이는 로봇 시스템의 고장 여부를 판단하는 과정에서 전류 변화와 전력 소비 패턴이 핵심적인 역할을 한다는 점을 시사한다.

또한 SMOTE와 SMOTE에 언더샘플링을 적용시킨 결과를 비교한 결과, 두 방법 모두 유사한 분류 성능을 보였다. SMOTE + 언더샘플링은 과적합을 소폭 감소시키는 효과를 보였으나, Failure 클래스의 Recall과 F1-score는 오히려 약간 감소하였다. ROC-AUC와 Precision-Recall Curve 역시 거의 동일한 수준으로 나타나, 본 데이터셋에서는 ENN을 통한 추가적인 노이즈 제거가 성능 향상에 크게 기여하지 못한 것으로 판단된다.

종합하면 Random Forest는 본 프로젝트에서 실험한 모델 중 가장 우수한 성능을 보였으며, 높은 정확도와 ROC-AUC뿐만 아니라 실제 고장 데이터를 안정적으로 탐지하는 능력을 확인할 수 있었다.

## 7. K-Nearest Neighbors

앞선 Random Forest 모델을 통해 복잡한 비선형 데이터에 대한 앙상블 기법의 강력함을 확인하며 현재까지 가장 우수한 분류 성능을 도출해냈다.

이제 마지막으로 거리 기반 알고리즘인 K-Nearest Neighbors를 적용해본다. 수백 개의 트리가 다수결 투표를 하는 RF보다 단순한 거리 기반의 KNN이 더 높은 성능을 돌파하기는 현실적으로 어려울 수 있다. 하지만 본 데이터셋은 EDA와 PCA 시각화에서 확인했듯 정상과 고장 데이터가 상당 부분 중첩되어 있다. 따라서 데이터의 기하학적 분포와 스케일에 전적으로 의존하는 KNN이 이 복잡한 공간 속에서 고장 데이터의 국소적 이웃 패턴을 어떻게 인식해내는지를 확인하는 것은, 데이터의 공간적 특성을 재검증하는 차원에서 충분한 분석적 의미를 가진다.



```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Imblearn Pipeline 구축 (SMOTE -> KNN)
pipeline = ImbPipeline([
    ('smote', SMOTE(random_state=42)),
    ('knn', KNeighborsClassifier())
])

# 파라미터 그리드
param_grid = {
    'knn__n_neighbors': [3, 5, 7, 9, 11], # 주변 몇 명을 확인할 것인가
    'knn__weights': ['uniform', 'distance'], # 가까운 놈한테 더 가중치를 줄 것인가
    'knn__metric': ['euclidean', 'manhattan'] # 거리 측정 공식
}

print("--- [SMOTE] KNN 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=skf,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train_scaled, y_train)

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search.best_params_.items():
    print(f" • {param.replace('knn__', '')}: {value}")
print("")
print(f"최고 교차 검증 F1-Macro 점수: {grid_search.best_score_:.4f}")

```

Standard Scaling을 한 뒤, SMOTE를 적용하고 grid search CV를 진행한다.

--- [모델 비교] 상위 10개 KNN 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정           |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|-------------------|
| 0 | 1  | 0.6934 | 1.0000   | 0.7183  | 0.2817  | 1.000000  | 0.901428 | 0.846840     | N=3, W=dis, M=man |
| 1 | 2  | 0.6922 | 0.8607   | 0.7169  | 0.1437  | 0.952583  | 0.899388 | 0.845806     | N=3, W=uni, M=man |
| 2 | 3  | 0.6893 | 0.8553   | 0.7131  | 0.1422  | 0.950204  | 0.898029 | 0.848348     | N=3, W=uni, M=euc |
| 3 | 4  | 0.6891 | 1.0000   | 0.7183  | 0.2817  | 1.000000  | 0.901428 | 0.849713     | N=3, W=dis, M=euc |
| 4 | 5  | 0.6804 | 1.0000   | 0.7179  | 0.2821  | 1.000000  | 0.891230 | 0.872278     | N=5, W=dis, M=euc |
| 5 | 6  | 0.6799 | 1.0000   | 0.7171  | 0.2829  | 1.000000  | 0.893950 | 0.885396     | N=5, W=dis, M=man |
| 6 | 7  | 0.6741 | 0.8016   | 0.7102  | 0.0914  | 0.923351  | 0.885792 | 0.870741     | N=5, W=uni, M=euc |
| 7 | 8  | 0.6718 | 0.8090   | 0.7086  | 0.1004  | 0.927430  | 0.889191 | 0.883673     | N=5, W=uni, M=man |
| 8 | 9  | 0.6718 | 1.0000   | 0.7187  | 0.2813  | 1.000000  | 0.886472 | 0.880339     | N=7, W=dis, M=euc |
| 9 | 10 | 0.6702 | 1.0000   | 0.7137  | 0.2863  | 1.000000  | 0.887152 | 0.900621     | N=7, W=dis, M=man |

distance로 가중치를 준 방식은 전부 Train F1이 1이 나왔는데 이는 distance 방식이 가까운 이웃에게 무한대의 가중치를 주기 때문이다. 즉, 학습데이터를 완벽하게 암기한 상태이기 때문에 1이 나왔다.

모델들의 Test F1은 평균적으로 0.718정도로 나왔다. 예상대로 RF보다 적게 나왔지만 SVM보다도 조금 낮다.

이유를 추측하자면 현재 사용하는 피쳐는 26개가 있다. RF는 알아서 전류와 파워를 뽑아와서 기준을 세웠지만 KNN은 우직하게 26개 변수 거리를 전부 똑같은 비중으로 계산했기 때문이다. 중요도가 낮은 변수들이 전류와 관련된 변수들의 거리를 덮어버려서 왜곡이 진행되어 차원의 저주가 일어난것이다.

```
[선택된 2위 모델의 하이퍼파라미터]
• metric: manhattan
• n_neighbors: 3
• weights: uniform

[Train 데이터 평가]
Accuracy (정확도) : 0.9526
F1-Macro Score   : 0.860673
ROC-AUC Score    : 0.999567

[Train 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 1.00 | 0.95 | 0.97 | 5470 |
| 1 (Failure) | 0.60 | 1.00 | 0.75 | 414 |
|---|---|---|---|---|
| | accuracy | | | 0.95 | 5884 |
| | macro avg | 0.80 | 0.97 | 0.86 | 5884 |
| | weighted avg | 0.97 | 0.95 | 0.96 | 5884 |

[Test 데이터 평가]
Accuracy (정확도) : 0.8994
F1-Macro Score   : 0.716924
ROC-AUC Score    : 0.845806

[Test 상세 분류 리포트]
| | | | | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 (Normal) | 0.97 | 0.92 | 0.94 | 1367 |
| 1 (Failure) | 0.38 | 0.68 | 0.49 | 104 |
|---|---|---|---|---|
| | accuracy | | | 0.90 | 1471 |
| | macro avg | 0.68 | 0.80 | 0.72 | 1471 |
| | weighted avg | 0.93 | 0.90 | 0.91 | 1471
```

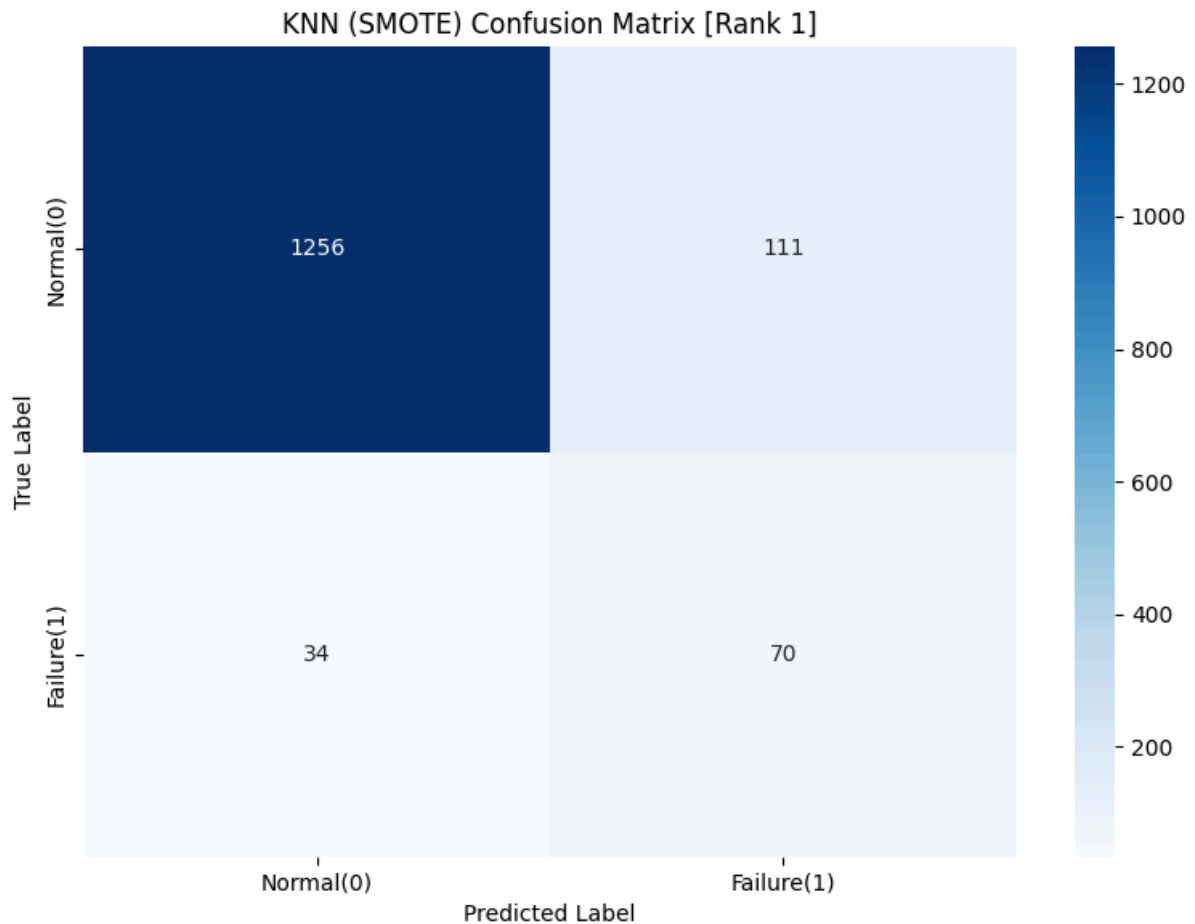
과적합을 0.14 수준으로 막아내고 Test F1도 0.7169인 2위 모델을 보면

Precision은 0.38,

Recall은 0.68,

F1 score는 0.49,

ROC-AUC는 0.84 수준으로 LR보다는 훨씬 높지만 SVM이나 RF보다 확실히 낮다.



KNN의 오차행렬 분석 결과 RF보다 약 2.5배나 많은 FP를 기록했다.

이러한 이유 중 하나는 보이지 않는 공간에 SMOTE가 만들어낸 가짜 고장 데이터들이 정상 데이터 주변에 있어 오탐지가 발생한다.

--- [SMOTE+ENN] KNN 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---  
 --- [모델 비교] 상위 10개 KNN (SMOTE+ENN) 성능 종합 ---

|   | 순위 | CV F1  | Train F1 | Test F1 | 과적합 Gap | Train Acc | Test Acc | Test ROC-AUC | 파라미터 설정           |
|---|----|--------|----------|---------|---------|-----------|----------|--------------|-------------------|
| 0 | 1  | 0.6974 | 0.9961   | 0.7257  | 0.2704  | 0.998980  | 0.908226 | 0.843586     | N=3, W=dis, M=man |
| 1 | 2  | 0.6967 | 0.8662   | 0.7241  | 0.1421  | 0.955472  | 0.906186 | 0.843484     | N=3, W=uni, M=man |
| 2 | 3  | 0.6942 | 0.8636   | 0.7137  | 0.1499  | 0.954453  | 0.900748 | 0.841216     | N=3, W=uni, M=euc |
| 3 | 4  | 0.6937 | 0.9954   | 0.7190  | 0.2764  | 0.998810  | 0.904147 | 0.842243     | N=3, W=dis, M=euc |
| 4 | 5  | 0.6855 | 0.9954   | 0.7259  | 0.2695  | 0.998810  | 0.898708 | 0.874325     | N=5, W=dis, M=euc |
| 5 | 6  | 0.6851 | 0.9961   | 0.7189  | 0.2772  | 0.998980  | 0.897349 | 0.877606     | N=5, W=dis, M=man |
| 6 | 7  | 0.6786 | 0.8090   | 0.7178  | 0.0912  | 0.928280  | 0.893270 | 0.872939     | N=5, W=uni, M=euc |
| 7 | 8  | 0.6768 | 0.8134   | 0.7139  | 0.0996  | 0.930659  | 0.893950 | 0.875995     | N=5, W=uni, M=man |
| 8 | 9  | 0.6761 | 0.9954   | 0.7140  | 0.2814  | 0.998810  | 0.888511 | 0.879871     | N=7, W=dis, M=euc |
| 9 | 10 | 0.6754 | 0.9954   | 0.7199  | 0.2755  | 0.998810  | 0.892590 | 0.900558     | N=7, W=dis, M=man |

SMOTE에 언더샘플링을 적용해 경계선 주변의 데이터들을 지워보았다.

점수가 소폭 상승하며 가장 정상적인 모델인 2위 모델의 점수가 더 0.7241로 더 올랐다.

--- [상세 분석] 2위 모델 최종 평가지표 및 시각화 ---

[Test 데이터 평가]

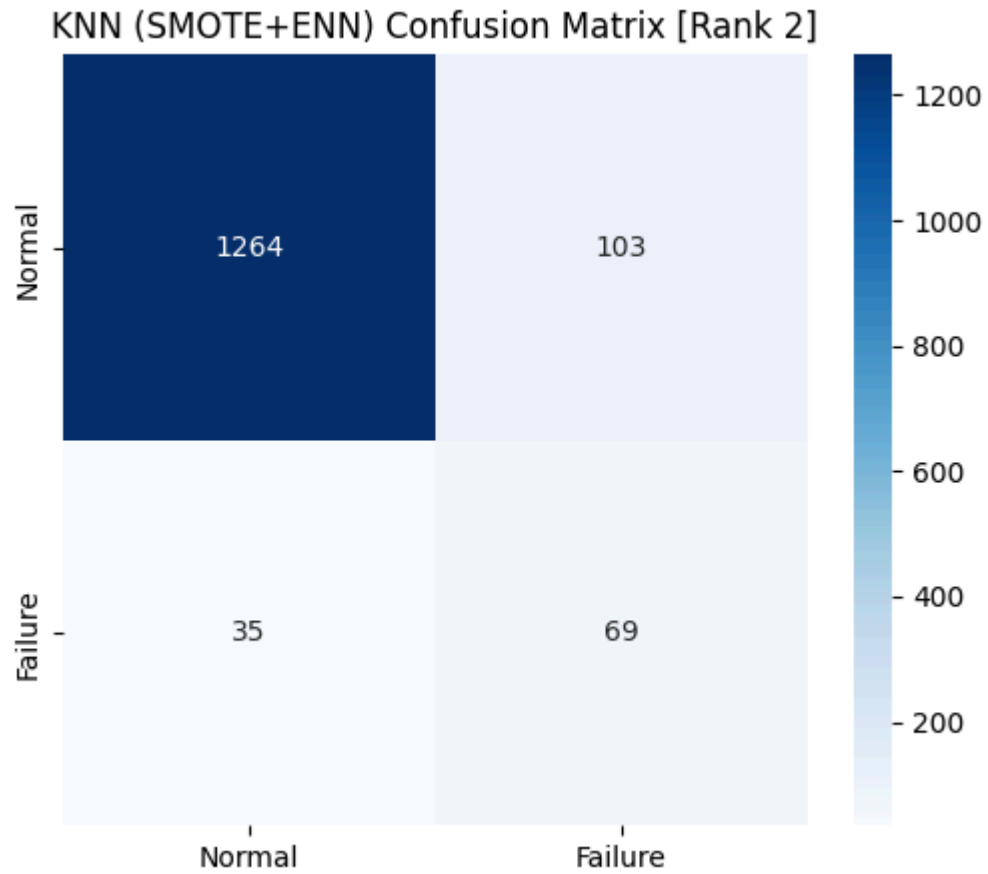
Accuracy : 0.9062

F1-Macro : 0.724119

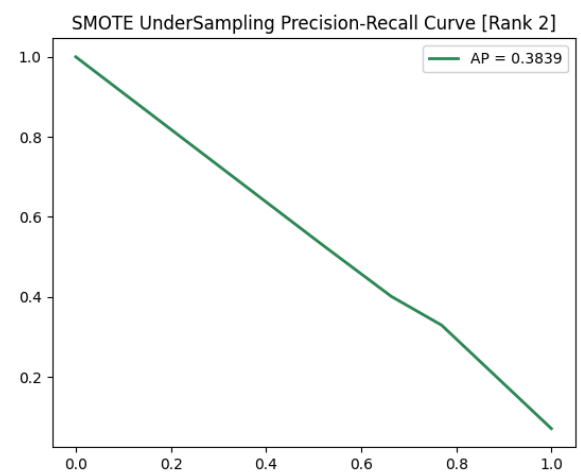
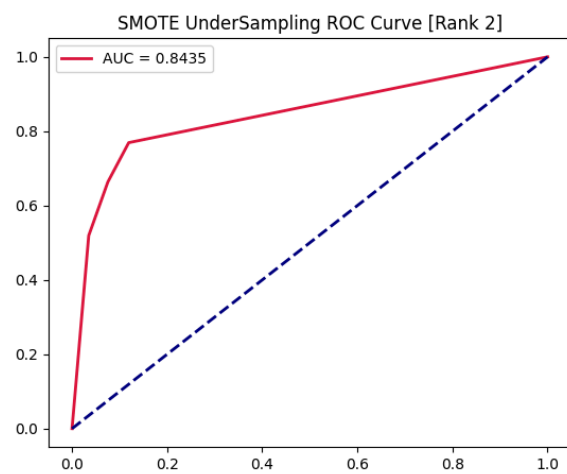
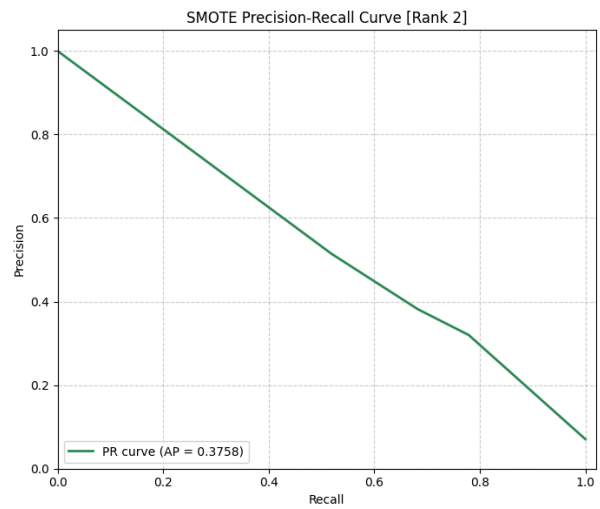
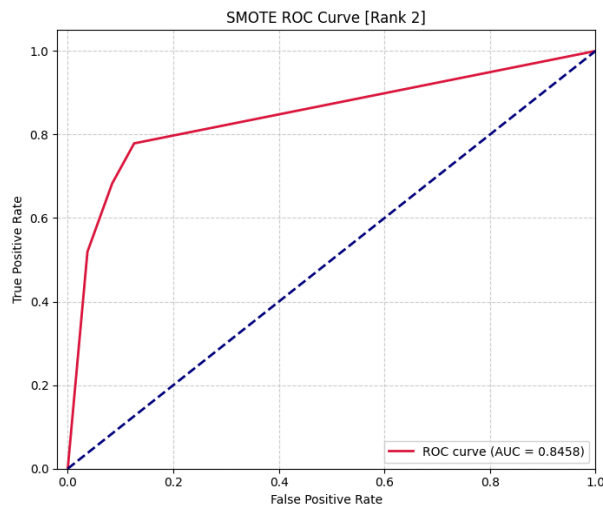
ROC-AUC : 0.843484

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0 (Normal)   | 0.97      | 0.92   | 0.95     | 1367    |
| 1 (Failure)  | 0.40      | 0.66   | 0.50     | 104     |
| accuracy     |           |        | 0.91     | 1471    |
| macro avg    | 0.69      | 0.79   | 0.72     | 1471    |
| weighted avg | 0.93      | 0.91   | 0.92     | 1471    |

언더샘플링 후, 2위 모델의 지표를 보면 Precision이 0.02 올랐고 F1 score가 0.01 올랐지만 Recall이 0.02 줄어들었다.



가짜 고장 데이터들이 사라지자 **TP**를 1건 양보하고 정상을 고장으로 오해하는 **FP**를 8건이나 줄이는 것에 성공했다.



ROC curve를 보면 각각 0.8458, 0.8435로 꽤 괜찮게 분류를 진행한다고 할 수 있다.

언더샘플링을 진행한 결과가 오히려 일반 SMOTE 보다 0.002점이 떨어졌다.

오른쪽 PR curve를 보면 각각 0.3758, 0.3839로 확실히 SMOTE 언더샘플링을 한 모델이 정밀도와 재현율의 관계를 더 잘 방어한다.

8. 결론

모델별 성능 비교

| 모델                           | Test Macro F1 | Accuracy    | ROC-AUC     | 특징                             |
|------------------------------|---------------|-------------|-------------|--------------------------------|
| Logistic Regression          | 0.51          | 0.66        | 0.77        | 선형 분류 한계, False Positive 다수 발생 |
| Support Vector Machine (RBF) | 0.73          | 0.90        | 0.86        | 비선형 패턴 학습 성공, FP 크게 감소         |
| Decision Tree                | 약 0.70~0.75   | 약 0.90      | 약 0.88      | 해석이 쉽지만 과적합 위험 존재              |
| K-Nearest Neighbors          | 0.72          | 0.90        | 0.85        | 준수한 성능, 그러나 과적합 경향 존재          |
| Random Forest (SMOTE)        | <b>0.80</b>   | <b>0.95</b> | <b>0.94</b> | 가장 우수한 성능과 안정성                 |
| Random Forest (SMOTE+ENN)    | 0.80          | 0.95        | 0.94        | SMOTE 대비 유의미한 성능 향상 없음         |

위 결과를 종합하면 Random Forest가 모든 평가 지표에서 가장 우수한 성능을 보였다. 특히 Test Macro F1 Score 0.80, Accuracy 94.63%, ROC-AUC 0.94를 기록하며 다른 모델들을 크게 앞섰다.

Classification Task에서는 UR3 협동로봇의 센서 데이터를 활용하여 System Failure를 예측하는 이진 분류 모델을 구축하였다. 데이터는 정상 93%, 고장 7%로 심한 클래스 불균형을 가지고 있었기 때문에 Accuracy만으로는 모델을 평가하기 어려웠으며, 이를 보완하기 위해 SMOTE를 적용하고 F1-Score, Recall, ROC-AUC를 중심으로 성능을 평가하였다.

EDA 결과 System Failure와 각 변수 간의 선형 상관관계는 매우 낮게 나타났으며, 정상과 고장 데이터의 분포 또한 상당 부분 중첩되어 있었다. 이는 단순한 선형 모델만으로는 고장 여부를 구분하기 어렵다는 점을 시사하였다. 또한 온도, 전류, 전력 변수에서 정상과 고장 데이터의 분포 차이가 일부 확인되어 비선형 패턴이 존재할 가능성을 확인할 수 있었다.

실험 결과 **Logistic Regression**은 **Test Macro F1** 약 **0.51** 수준에 머물렀으며, 정상 데이터를 고장으로 오인하는 **False Positive**가 매우 많이 발생하였다. 반면 **RBF** 커널을 사용한 **SVM**은 **Test Macro F1** 약 **0.73**, **Accuracy** 약 **90%**, **ROC-AUC** 약 **0.86**을 기록하며 선형 모델보다 훨씬 우수한 성능을 보였다. 이는 본 데이터셋이 단순한 선형 구조가 아니라 비선형 구조를 포함하고 있음을 보여주는 결과였다.

**Decision Tree**와 **Random Forest**는 이러한 비선형 구조를 더욱 효과적으로 학습하였다. 특히 **SMOTE**를 적용한 **Random Forest**는 **Test Accuracy 94.63%**, **Macro F1 Score 0.80**, **ROC-AUC 0.94**를 기록하며 가장 우수한 성능을 보였다. 또한 실제 고장 데이터의 약 **65%**를 탐지하면서도 높은 정확도를 유지하였다. **Feature Importance** 분석 결과 **Current J3**, **Current J5**, **Current J2**, **Tool\_current** 등의 전류 관련 변수들이 가장 중요한 변수로 나타났으며, 이는 로봇 시스템의 고장 여부가 전류 및 전력 소비 패턴과 밀접하게 관련되어 있음을 시사한다.

추가적으로 **SMOTE**와 **SMOTE**로 생성된 경계선의 데이터를 지우기 위해 언더샘플링을 도입한 결과를 비교하면, **Logistic Regression**에서는 일부 성능 향상이 나타났지만 **Random Forest**에서는 **ROC-AUC**, **Precision-Recall Curve**, **Macro F1 Score**가 거의 동일하게 나타났다. 따라서 본 데이터셋에서는 **ENN**을 통한 추가적인 노이즈 제거가 큰 성능 향상으로 이어지지 않았다.

또한 **K-Nearest Neighbors(KNN)**은 **Test Macro F1** 약 **0.72** 수준으로 **SVM**과 유사한 성능을 보였으나, 학습 데이터에서는 **F1 Score**가 **1.0**에 가까워 과적합 경향이 크게 나타났다. 반면 **Random Forest**는 더 높은 일반화 성능과 안정적인 **ROC-AUC**를 보여 최종 모델로 선택하기에 가장 적합하였다.



종합적으로 본 데이터셋은 선형 모델보다 비선형 모델이 훨씬 적합하였으며, 여러 센서 변수 간의 복합적인 상호작용을 학습할 수 있는 **Random Forest**가 가장 우수한 성능을 보였다.